



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 5

Path Planning

Session Faculty:

Raja vadhana Prabhakar

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

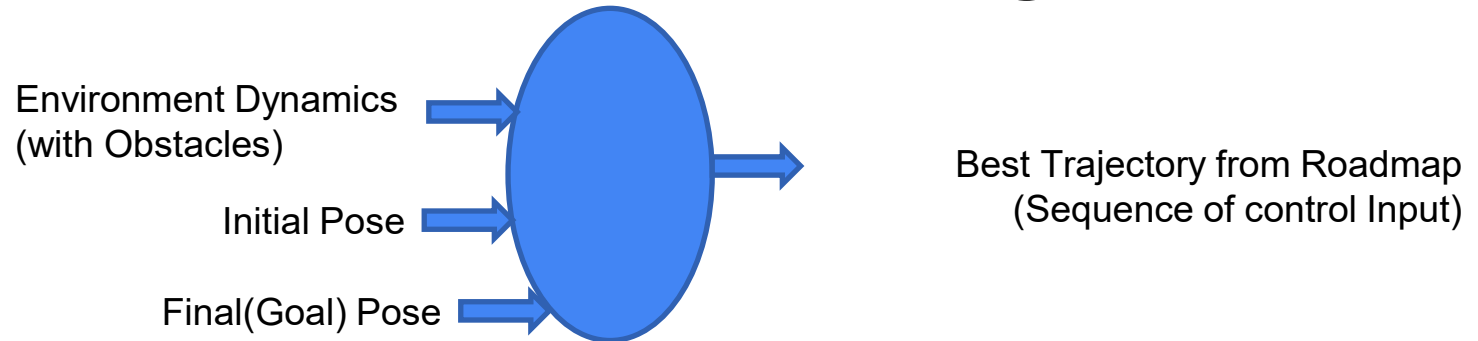
☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutionary Techniques

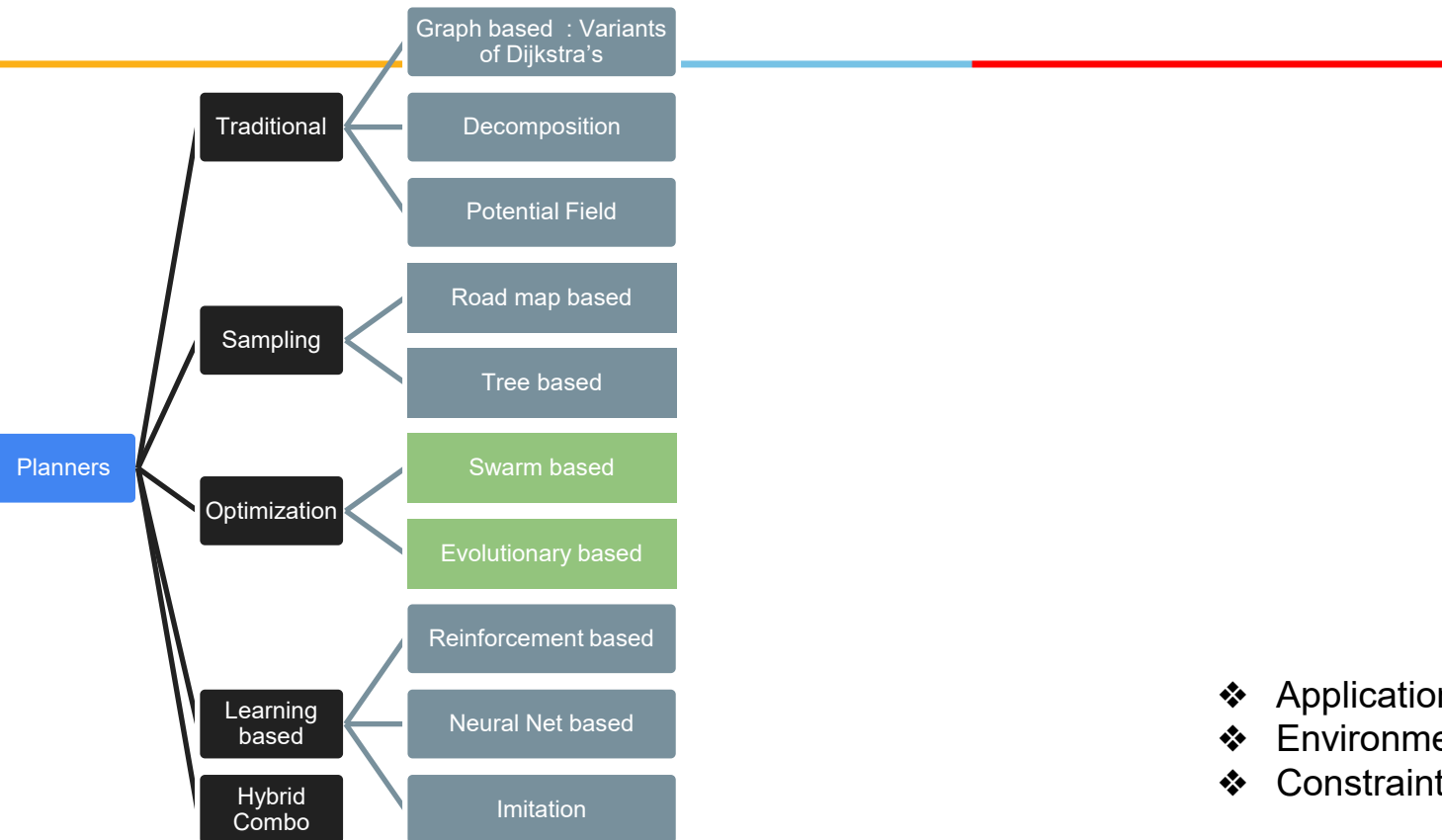
☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

Robot Path Planning



Taxonomy of Planning Problems



- ❖ Application
- ❖ Environment
- ❖ Constraints

Optimization algorithm based planning



These planners treat path planning as a numerical optimization problem. They start with an initial path and iteratively improve it.

- **How they work:** They define a trajectory as a set of points and create a cost function that measures path quality (e.g., length, smoothness, clearance from obstacles). The algorithm then adjusts the points in the trajectory to minimize this cost while satisfying constraints (like avoiding collisions).
- **Applications:**
 - **Drone Acrobatics:** Generating the smooth, dynamic trajectories needed for high-speed flight.
 - **Robotic Manipulation:** Creating precise and smooth movements for tasks like assembly or welding.
- **Pros & Cons:**
 - **High-Quality Paths**
 - **Local Minima**
 - **Requires Initialization**

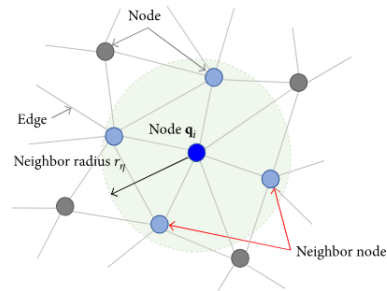
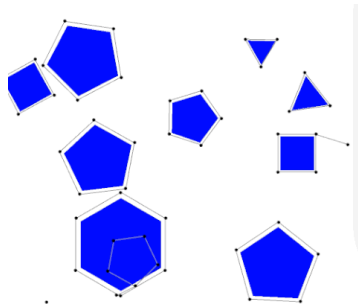
Optimization-based path planning treats motion planning as a **continuous optimization problem**:

- start with an **initial path guess** (could be straight line or a path from another planner like PRM/A*).
- **optimize** this path with respect to a **cost function** subject to **constraints**.
- Output is a **smooth, dynamically feasible trajectory**.

Analogy: Instead of searching node-by-node like A*, you **mold an initial path into the best possible path** based on physics, smoothness, and safety.

Nature Inspired Planners

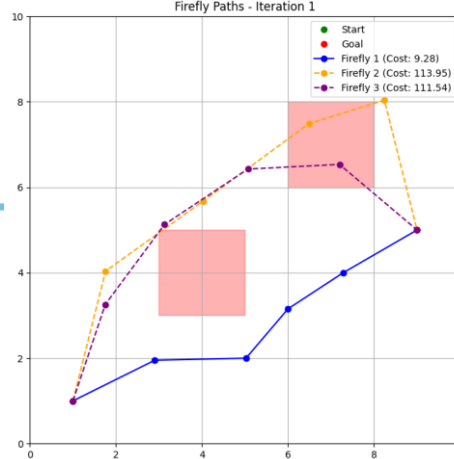
Firefly Algorithm





FireFly Algorithm

1. Initialize population
2. Evaluate the fitness



```

Initialize firefly population and light intensity based on objective function
Define light absorption coefficient  $\gamma$ 
Loop while generations are less than MaxGeneration
    For each firefly i and j:
        Calculate distance r
        If firefly j is brighter than i:
            Calculate attractiveness  $\beta$ 
            Move firefly i towards j
        End if
    End for j
    Evaluate new solutions and update light intensity
End for i
Rank fireflies and find the best solution
Increment generation
End loop
    
```

Firefly 1: [(1,1), (3,2), (6,4), (9,8), (10.5,10)]

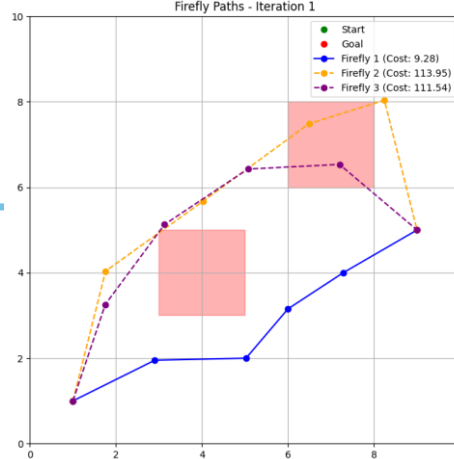
Firefly 2: [(1,1), (2,5), (5,6), (8,9), (10.5,10)]

Firefly 3: [(1,1), (4,3), (**6.5,7**), (9.5,9), (10.5,10)]



FireFly Algorithm

1. Initialize population
2. Evaluate the fitness
3. Choose the brightest



```

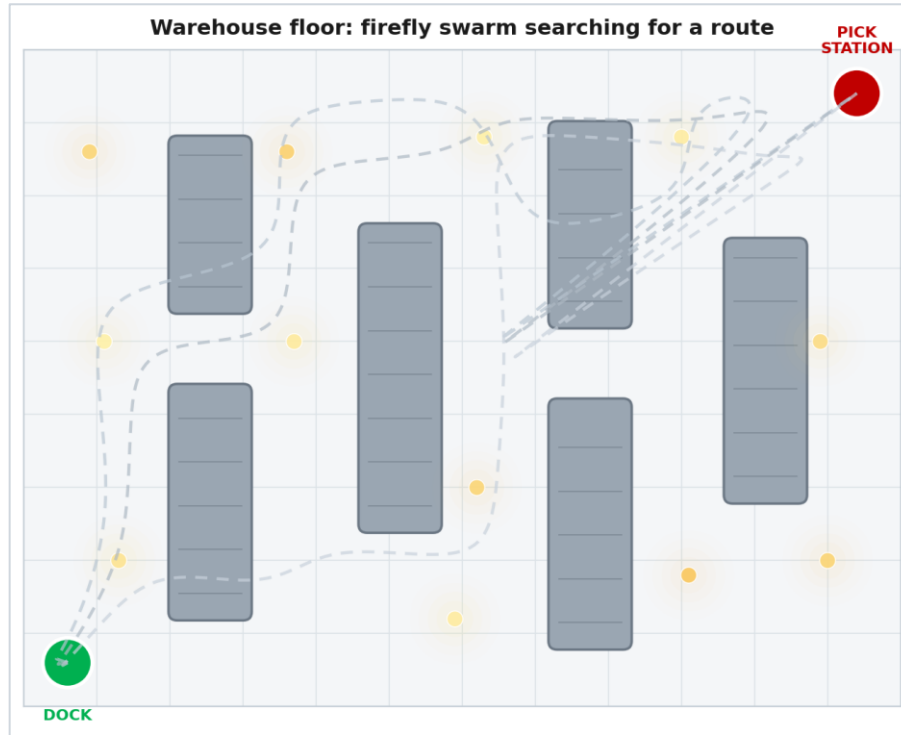
Initialize firefly population and light intensity based on objective function
Define light absorption coefficient  $\gamma$ 
Loop while generations are less than MaxGeneration
    For each firefly i and j:
        Calculate distance r
        If firefly j is brighter than i:
            Calculate attractiveness  $\beta$ 
            Move firefly i towards j
        End if
    End for j
    Evaluate new solutions and update light intensity
End for i
Rank fireflies and find the best solution
Increment generation
End loop
    
```

Firefly 1: [(1,1), (3,2), (6,4), (9,8), (10.5,10)]

Firefly 2: [(1,1), (2,5), (5,6), (8,9), (10.5,10)]

Firefly 3: [(1,1), (4,3), (6.5,7), (9.5,9), (10.5,10)]

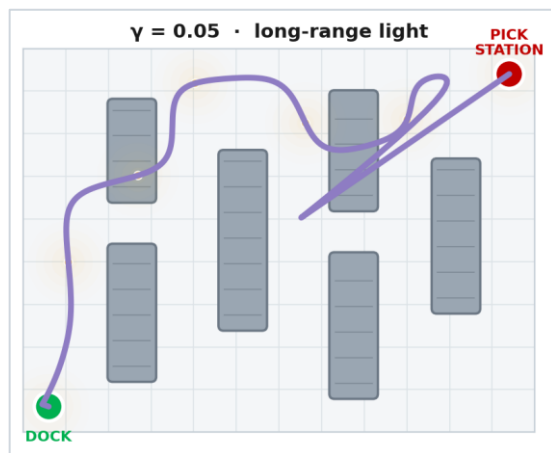
Firefly Algorithm : Warehouse Use case



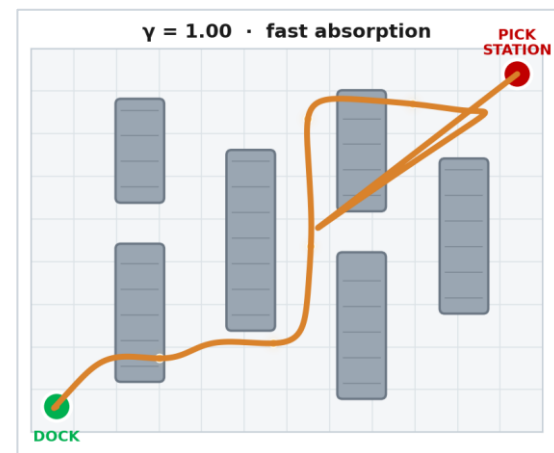
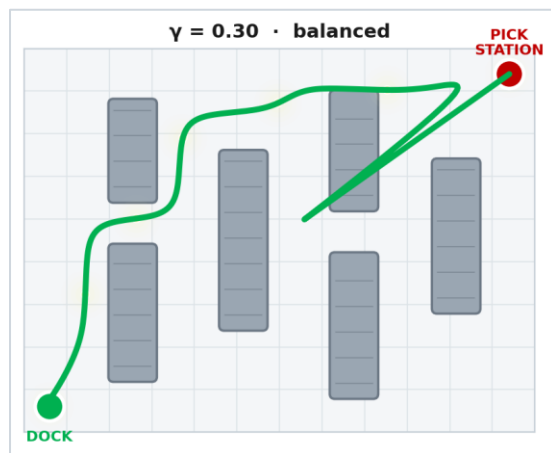
- Each firefly is one candidate route from the Dock to the Pick Station.
- Brightness = route quality — brighter means lower path cost.
- Fireflies move toward brighter neighbors, refining the route each step.
- Light absorption γ controls how far that attraction reaches.

Hyperparameter — Light Absorption γ

◀ Low γ (0.05)



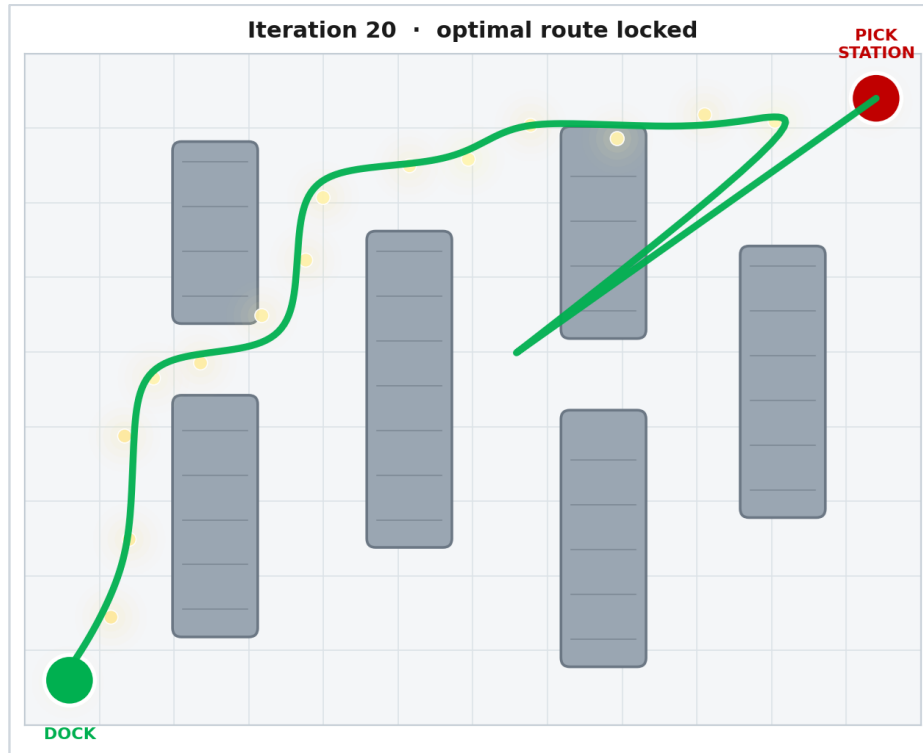
High γ (1.00) ▶



Low γ — light travels far → wide exploration but wavy, longer paths. **High γ** — light absorbed fast → short-sighted, jagged, poor clearance.

$\gamma \approx 0.30$ balances exploration and refinement → the shortest, smoothest route.

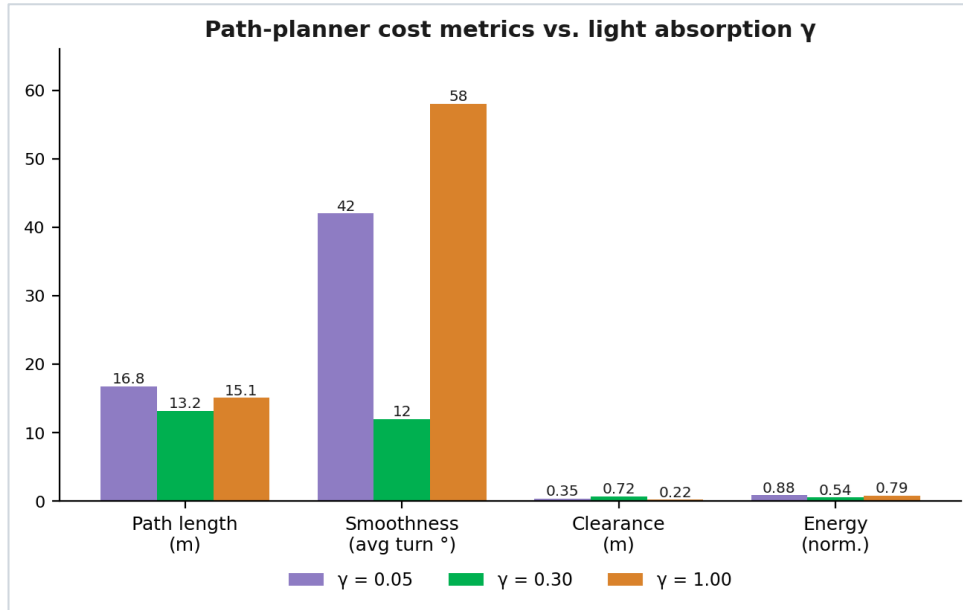
Sample Convergence



- Frame 1 — scattered search, path cost high.
- Frame 2 — swarm converges, cost dropping fast.
- Frame 3 — optimal route locked in green.

Brighter fireflies pull the swarm onto the lowest-cost route

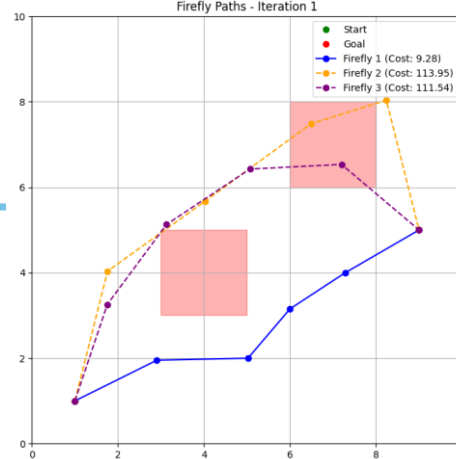
Additional Performance Metrics - Idea



Balanced γ (0.30) wins on every cost: -21% path length · 3.5× smoother · 2× obstacle clearance vs. a poorly-tuned γ .

FireFly Algorithm

1. Initialize population
2. Evaluate the fitness
3. Choose the brightest
4. Move others based on attractiveness



```

Initialize firefly population and light intensity based on objective function
Define light absorption coefficient  $\gamma$ 
Loop while generations are less than MaxGeneration
    For each firefly i and j:
        Calculate distance r
        If firefly j is brighter than i:
            Calculate attractiveness  $\beta$ 
            Move firefly i towards j
        End if
    End for j
    Evaluate new solutions and update light intensity
End for i
Rank fireflies and find the best solution
Increment generation
End loop
    
```

Firefly 1: [(1,1), (3,2), (6,4), (9,8), (10.5,10)]

Firefly 2: [(1,1), (2,5), (5,6), (8,9), (10.5,10)]

Firefly 3: [(1,1), (4,3), (6.5,7), (9.5,9), (10.5,10)]

$\beta_0 = 1$

$\gamma = 0.1$

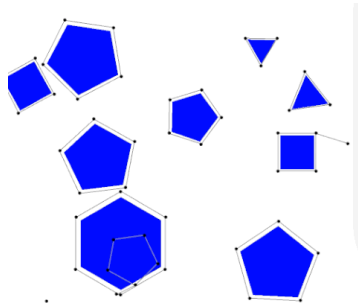
$\alpha = 0.2$

r = Euclidean distance

Rand = 0.3

FA helps provide smooth continuous cost surfaces where attraction gives faster early convergence

Optimization algorithms based Path Planning **Genetic Algorithm**



GA as path planning (way-point Optimization)



Population

Fitness
Calculation

Mating Pool

Parents Selection

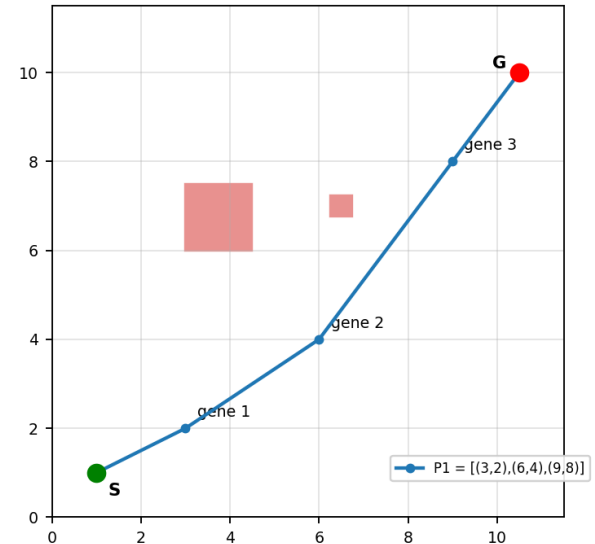
Mating (Crossover
& Mutation)

Offspring

Chromosome Design :

$S \rightarrow \text{gene1} \rightarrow \text{gene2} \rightarrow \text{gene3} \rightarrow G$

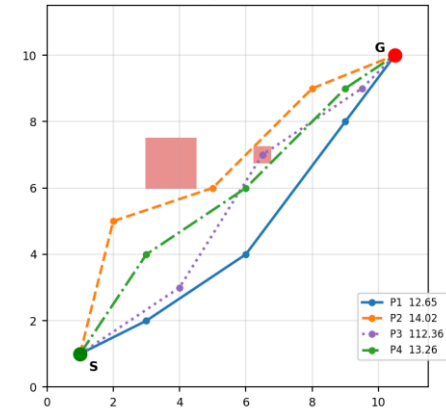
Chromosome = 3 via-points (P1 shown)



GA as path planning (way-point Optimization)

Hyperparameter : Population size (4 here, 50–200 in practice): larger population = better coverage of via-point space, higher cost per generation

Generation 0 population (4 individuals)



1. Initialize population
2. Evaluate the fitness
3. Choose the fittest (Roulette Wheel)
4. Evolve potentials
5. Ready next generation by replacement (Elitist)

Start S: (1, 1) **Goal G:** (10.5, 10)

Chromosome Design = Decision variable:

An ordered set of 3 intermediate way points between S and G.

- **Candidate paths :**

- P1: (1,1), (3,2), (6,4), (9,8), (10.5,10)
- P2: (1,1), (2,5), (5,6), (8,9), (10.5,10)
- P3: (1,1), (4,3), (6.5,7), (9.5,9), (10.5,10)
- P4: (1,1), (3,4), (6,6), (9,9), (10.5,10)

- **Goal of the GA:** Evolve the way-point set that minimizes path cost instead of moving fireflies by brightness.

GA as path planning (way-point Optimization)



Population



Fitness
Calculation



Mating Pool



Parents Selection



Mating (Crossover
& Mutation)



Offspring

1. Initialize population
2. **Evaluate the fitness**
3. Choose the fittest (Roulette Wheel)
4. Evolve potentials
5. Ready next generation by replacement (Elitist)

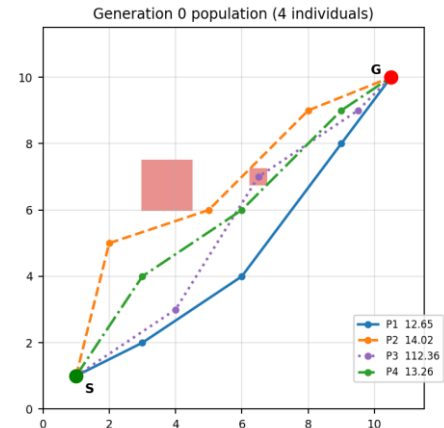
Start S: (1, 1) **Goal G:** (10.5, 10)

Candidate paths :

- P1: (1,1), (3,2), (6,4), (9,8), (10.5,10) → L = 12.65
- P2: (1,1), (2,5), (5,6), (8,9), (10.5,10) → L = 14.02
- P3: (1,1), (4,3), (6.5,7), (9.5,9), (10.5,10) → L = 12.36, collides ⇒ Cost = 112.36
- P4: (1,1), (3,4), (6,6), (9,9), (10.5,10) → L = 13.26

$$\text{Fitness} = \text{Fitness}(P) = 1 / \text{Cost}(P)$$

$$\text{Cost}(P) = \sum \text{segment length} + 100 \times (\text{number of colliding segments})$$



GA as path planning (way-point Optimization)



Population



Fitness
Calculation



Mating Pool



Parents Selection



Mating (Crossover
& Mutation)



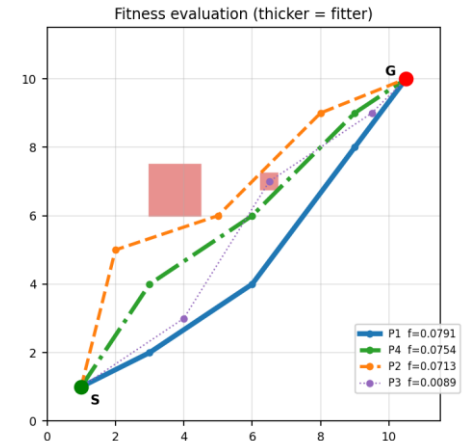
Offspring

1. Initialize population
2. Evaluate the fitness
3. **Choose the fittest (Roulette Wheel)**
4. Evolve potentials
5. Ready next generation by replacement (Elitist)

Start S: (1, 1) **Goal G:** (10.5, 10)

Candidate paths :

- P1: $\rightarrow L = 12.65 \rightarrow 0.0791 \rightarrow \mathbf{33.7 \%}$
- P2: $\rightarrow L = 14.02 \rightarrow 0.0713 \rightarrow \mathbf{30.4 \%}$
- P3: $\rightarrow L = 12.36, \text{collides} \Rightarrow \text{Cost} = 112.36 \rightarrow 0.0089 \rightarrow \mathbf{3.8 \%}$
- P4: $\rightarrow L = 13.26 \rightarrow 0.0754 \rightarrow \mathbf{32.1 \%}$



GA as path planning (way-point Optimization)



Population

Fitness
Calculation

Mating Pool

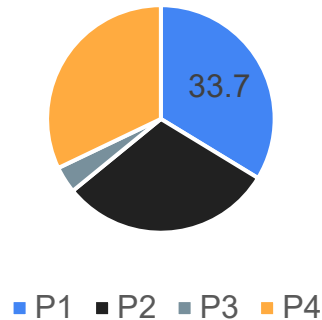
Parents Selection

Mating (Crossover
& Mutation)

Offspring

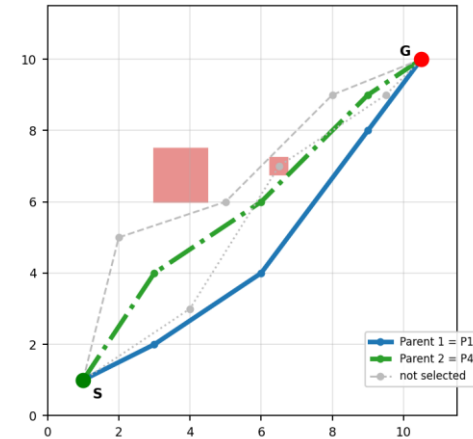
1. Initialize population
2. Evaluate the fitness
3. **Choose the fittest (Roulette Wheel)**
4. Evolve potentials
5. Ready next generation by replacement (Elitist)

Fitness



SPIN

Selection: parents P1 (33.7%) and P4 (32.1%)



Spin 1 $\rightarrow r = 0.21$ falls in $[0, 0.337) \Rightarrow$ **Parent 1 = P1**
Spin 2 $\rightarrow r = 0.88$ falls in $[0.679, 1.0) \Rightarrow$ **Parent 2 = P4**

GA as path planning (way-point Optimization)

FA moves a whole path toward a brighter path
GA recombines way points from two parent paths

Hyperparameter : Crossover rate (0.7–0.9): how often parents recombine instead of being copied.

Population

Fitness Calculation

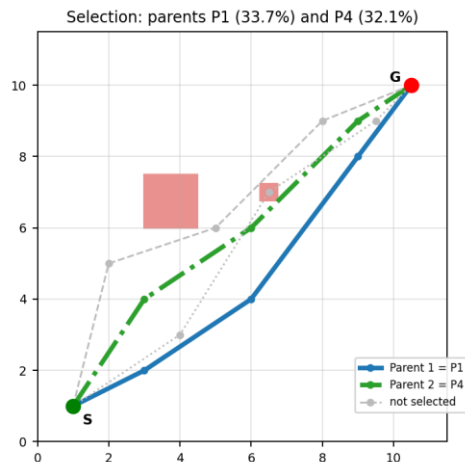
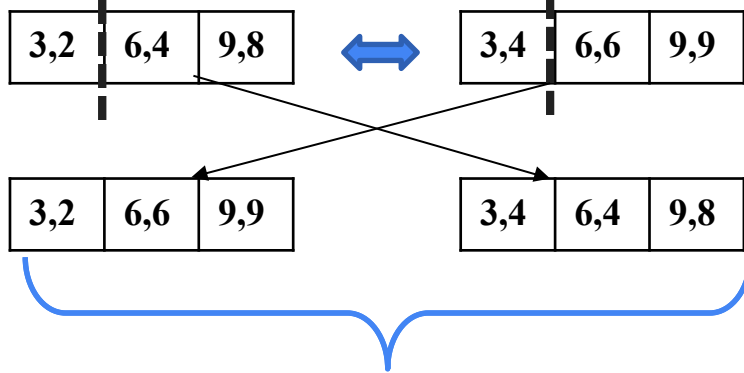
Mating Pool

Parents Selection

Mating (Crossover & Mutation)

Offspring

1. Initialize population
2. Evaluate the fitness
3. Choose the fittest (Roulette Wheel)
4. **Evolve potentials**
5. Ready next generation by replacement (Elitist)



Spin 1 $\rightarrow r = 0.21$ falls in $[0, 0.337) \Rightarrow$ **Parent 1 = P1**

Spin 2 $\rightarrow r = 0.88$ falls in $[0.679, 1.0) \Rightarrow$ **Parent 2 = P4**

GA as path planning (way-point Optimization)



Population



Fitness
Calculation



Mating Pool



Parents Selection



Mating (Crossover
& Mutation)



Offspring

1. Initialize population
2. Evaluate the fitness
3. Choose the fittest (Roulette Wheel)
4. **Evolve potentials**
5. Ready next generation by replacement (Elitist)

3,2	6,4	9,8
-----	-----	-----

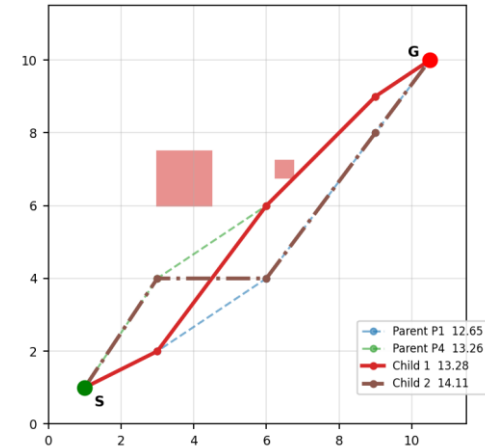
3,4	6,6	9,9
-----	-----	-----

3,2	6,6	9,9
-----	-----	-----

3,4	6,4	9,8
-----	-----	-----

- **Child 1:** (1,1)→(3,2)→(6,6)→(9,9)→(10.5,10) → L = **13.28**
- **Child 2:** (1,1)→(3,4)→(6,4)→(9,8)→(10.5,10) → L = **14.11**

Crossover after gene 1

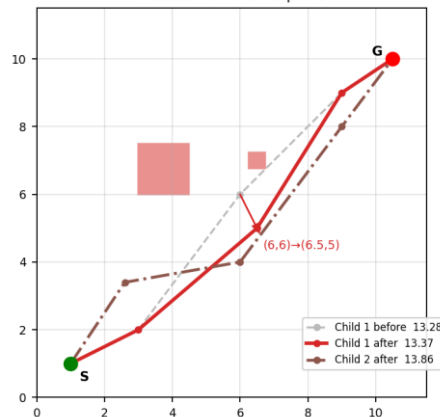


GA as path planning (way-point Optimization)

FA uses $\alpha(\text{rand} - 0.5)$ on every step
GA injects randomness only through mutation

Hyperparameter : Mutation rate (0.01–0.1 per gene) and step σ (0.5 units): σ controls how far a point can jump.

Mutation of one via-point



Gaussian perturbation of one gene: $x' = x + N(0, \sigma)$,

with $\sigma = 0.5$ grid units, one gene per offspring.

- **Child 1** gene 2 (6, 6) $\rightarrow$ sample (+0.5, -1.0) $\Rightarrow$ **(6.5, 5.0)**
 - New path: (1,1) $\rightarrow$ (3,2) $\rightarrow$ (6.5,5) $\rightarrow$ (9,9) $\rightarrow$ (10.5,10) : L = **13.37**
- **Child 2** gene 1 (3, 4) $\rightarrow$ sample (-0.4, -0.6) $\Rightarrow$ **(2.6, 3.4)** : L = **13.86**

Mutation is the only operator that can create way points never present in the initial population — essential for escaping obstacle-shaped local minima.

1. Initialize population
2. Evaluate the fitness
3. Choose the fittest (Roulette Wheel)
4. **Evolve potentials**
5. Ready next generation by replacement (Elitist)

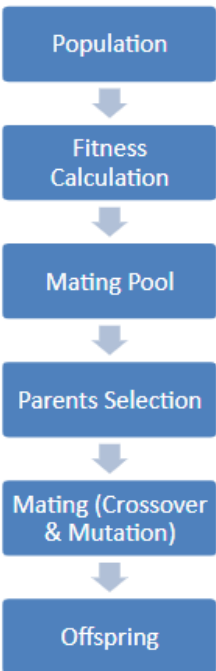
3,2	6,4	9,8
-----	-----	-----

3,4	6,6	9,9
-----	-----	-----

3,2	6,6	9,9
-----	-----	-----

3,4	6,4	9,8
-----	-----	-----

- **Child 1:** (1,1) $\rightarrow$ (3,2) $\rightarrow$ (6,6) $\rightarrow$ (9,9) $\rightarrow$ (10.5,10) $\rightarrow$ L = **13.28**
- **Child 2:** (1,1) $\rightarrow$ (3,4) $\rightarrow$ (6,4) $\rightarrow$ (9,8) $\rightarrow$ (10.5,10) $\rightarrow$ L = **14.11**



GA as path planning (way-point Optimization)

FA firefly keeps the brightest implicitly
GA keeps it explicitly through elitism.

Hyperparameter : Elitism count (1–2): guarantees best-cost improvement.

Population

Fitness
Calculation

Mating Pool

Parents Selection

Mating (Crossover
& Mutation)

Offspring

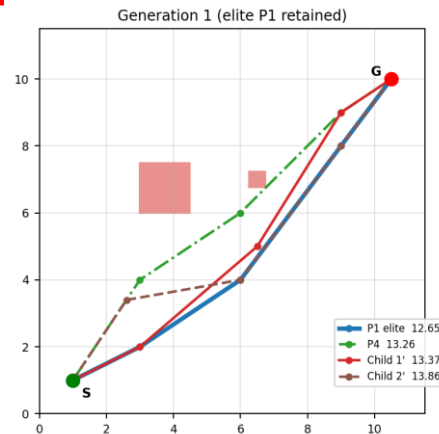
1. Initialize population
2. Evaluate the fitness
3. Choose the fittest (Roulette Wheel)
4. Evolve potentials
5. **Ready next generation by replacement (Elitist)**

3,2	6,4	9,8
-----	-----	-----

3,4	6,6	9,9
-----	-----	-----

3,2	6,6	9,9
-----	-----	-----

3,4	6,4	9,8
-----	-----	-----



- The elite individual **P1 (12.65)** is copied unchanged — the best-so-far path can never be lost.
- Average cost drops from 38.07 (Gen 0) to **13.29** (Gen 1) because the colliding individual has been evolved out.

Convergence over Generations



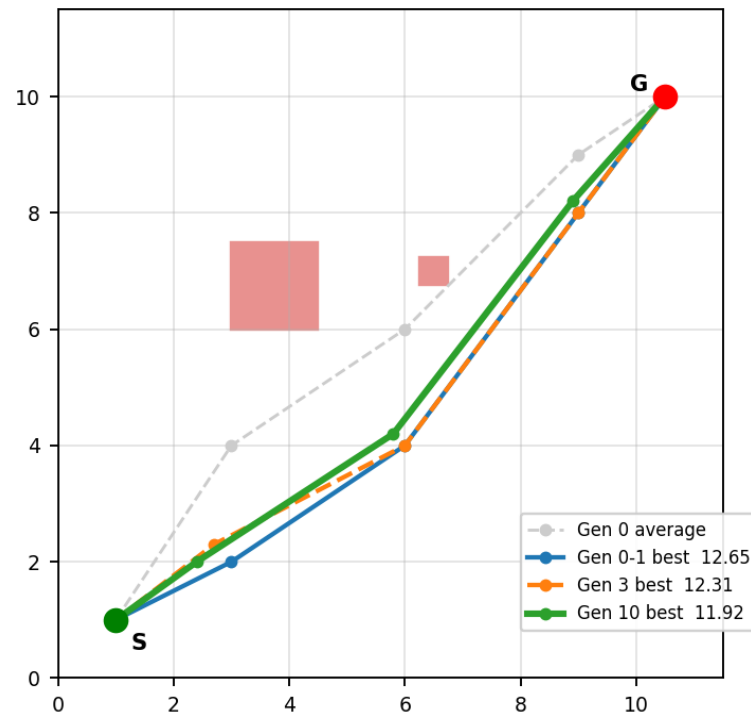
Hyperparameter : Termination: max generations or a stagnation window

Best-so-far cost (Lab Demo Python code)

- **Gen 0:** best 12.65 — average 38.07 (one colliding path)
- **Gen 1:** best 12.65 — average 13.29
- **Gen 3:** best 12.31 — via-point (3,2) drifts to (2.7,2.3)
- **Gen 6:** best 12.08 — middle point straightens toward the obstacle corner
- **Gen 10:** best **11.92**, population diversity collapsing — stop criterion met

GA helps work with discrete or mixed chromosome design, many constraints, easy parallel fitness evaluation.

Best-so-far path over generations



Mathematical Formulation



Let:

- $\mathbf{x}(t)$ = robot state at time t
- $u(t)$ = control input at time t
- $\mathcal{C}_{free}$ = collision-free configuration space

We minimize:

$$\min_{\mathbf{x}(t), u(t)} J(\mathbf{x}(t), u(t))$$

where J is **cost function** (e.g., path length, smoothness, energy).

Subject to:

1. **Dynamics constraints:**

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), u(t))$$

2. **Collision avoidance:**

$$\mathbf{x}(t) \in \mathcal{C}_{free}, \quad \forall t$$

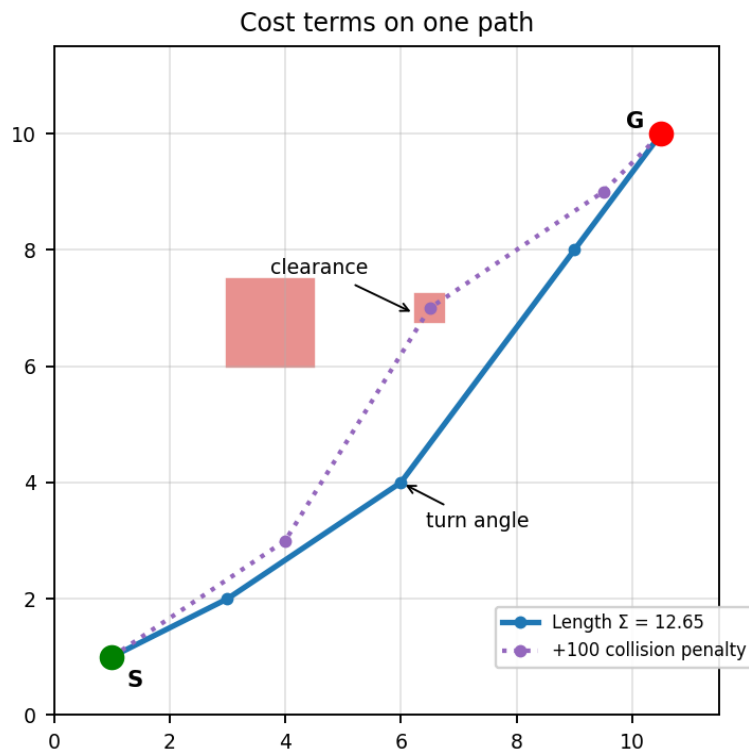
3. **Boundary conditions:**

$$\mathbf{x}(0) = \mathbf{x}_{start}, \quad \mathbf{x}(T) = \mathbf{x}_{goal}$$

4. **Control limits:**

$$u_{min} \leq u(t) \leq u_{max}$$

Cost Function Design



1. **Path length** – shorter is better:

$$J_{length} = \int_0^T \|\dot{\mathbf{x}}(t)\| dt$$

2. **Smoothness** – penalize high curvature or acceleration:

$$J_{smooth} = \int_0^T \|\ddot{\mathbf{x}}(t)\|^2 dt$$

3. **Obstacle clearance** – push trajectory away from obstacles:

$$J_{obs} = \sum_{p \in \text{path}} \phi(d(p, \text{obstacle}))$$

where ϕ increases sharply near collisions.

4. **Energy or control effort:**

$$J_{energy} = \int_0^T \|u(t)\|^2 dt$$

Total cost:

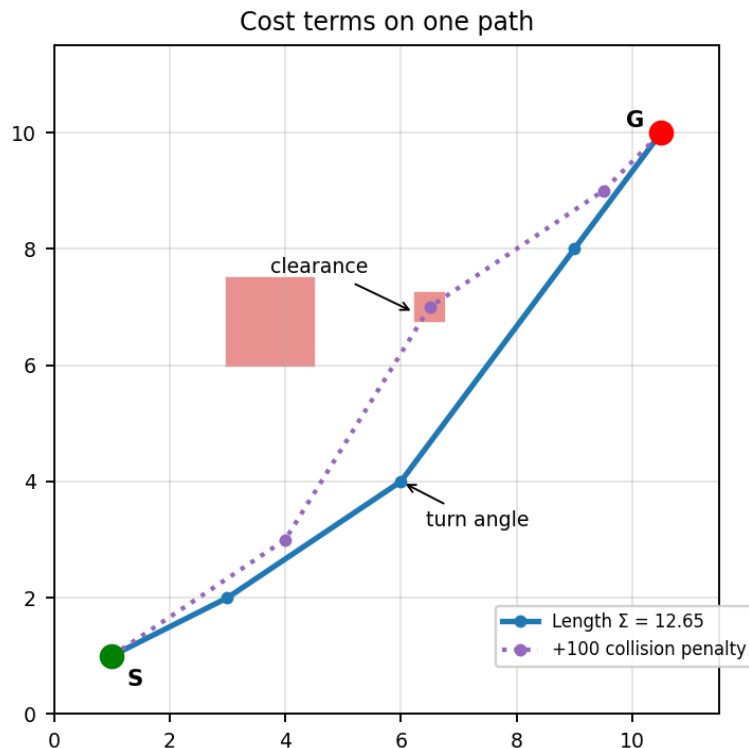
$$J = w_1 J_{length} + w_2 J_{smooth} + w_3 J_{obs} + w_4 J_{energy}$$

where w_i are tunable weights.

Cost Function Design



Cost function used in practice:



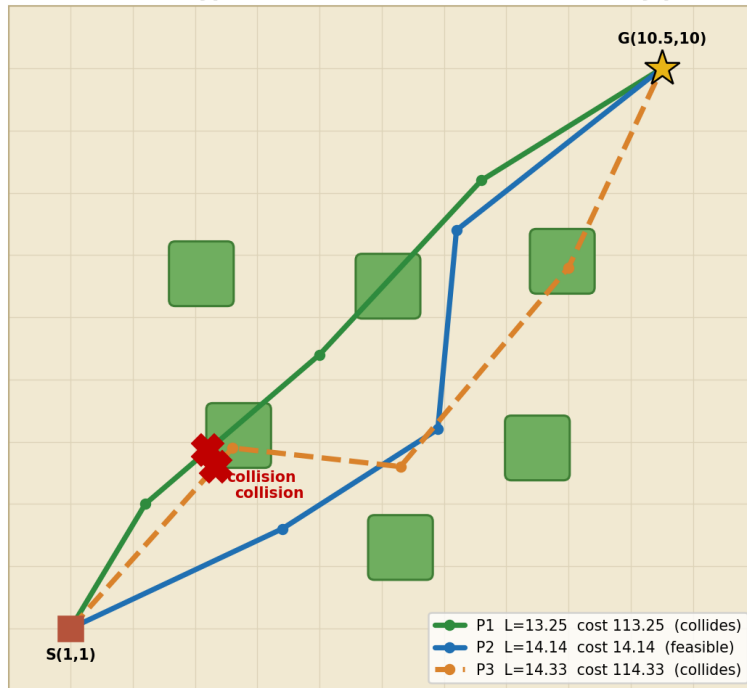
- **Cost** = $w_1 \cdot \text{Length} + w_2 \cdot \text{Collisions} + w_3 \cdot \text{Smoothness} + w_4 \cdot \text{Clearance}^{-1} + w_5 \cdot \text{Energy}$
- **Length**: Σ Euclidean distance between consecutive way points
- **Collision**: hard penalty, 100 per violating segment
- **Smoothness**: Σ turning angle between successive segments
- **Clearance**: inverse of minimum distance to the nearest obstacle
- **Energy / time**: weighted by velocity or terrain cost when the map is not uniform.

Normalize each term to a comparable range before weighting; set the collision weight at least an order of magnitude above the largest feasible path length; use **Fitness** = $1 / (1 + \text{Cost})$ to avoid divide by zero.

Live run – continuous waypoint GA (3 evolved way-points)



Continuous waypoint GA – evolve 3 intermediate way-points



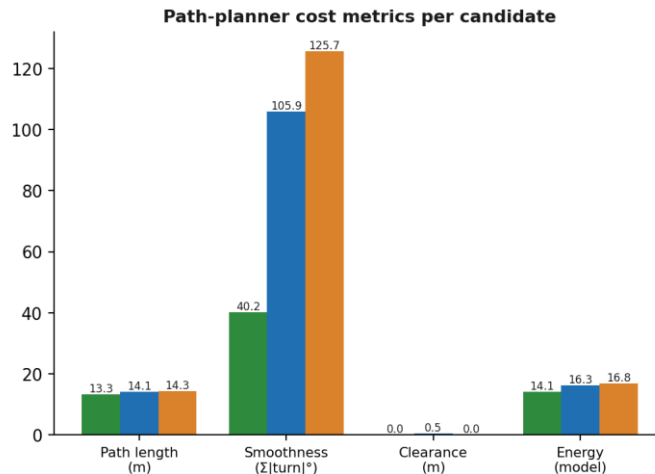
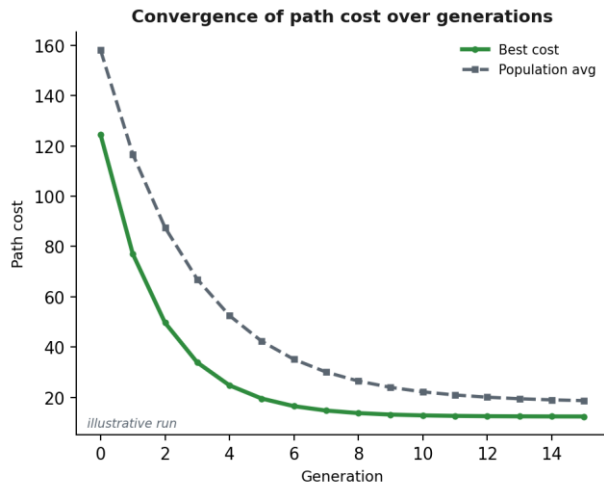
P2 feasible: clearance 0.47 m, cost 14.14. P1 & P3 clip a crop bed → +100 penalty. Cost = PathLen + 100·Collision.

Path-planner cost metrics & recommended energy model

innovate

achieve

lead



Recommended energy / effort model

$$E = \sum (e_t \cdot \Delta d + e_r \cdot |\Delta \theta| + e_p \cdot \text{slope} \cdot \Delta d)$$

e_t – translational cost per metre (rolling resistance)

e_r – rotational cost per degree of turn

e_p – terrain / slope penalty per metre

Simulate: sample the path, sum per-segment distance, turn angle and slope.

Eg., $e_t=1.0$, $e_r=0.02 \rightarrow E(P2)=16.3$ units (best feasible route).

```
Function GeneticAlgorithmPathPlanner:
// 1. Initialization
Population = CreateRandomPopulation(size, chromosome_length)

Loop for N generations:
// 2. Fitness Evaluation
For each Chromosome in Population:
    Path = DecodeChromosome(Chromosome)
    Cost = CalculateCost(Path, Start, Goal, Obstacles)
    Chromosome.fitness = 1 / Cost

// 3. Selection
Parents = SelectParents(Population) // e.g., Roulette Wheel

// 4. Crossover & Mutation
NewPopulation = []
While NewPopulation.size < Population.size:
    Parent1, Parent2 = pick from Parents
    Child1, Child2 = Crossover(Parent1, Parent2)
    Mutate(Child1)
    Mutate(Child2)
    Add Child1, Child2 to NewPopulation

// 5. Create Next Generation
Population = NewPopulation

// Return the best path found
BestChromosome = FindBest(Population)
Return DecodeChromosome(BestChromosome)
```

Additional Practice problem Added

Example #2



Treat a path as an individual ("chromosome") and evolve a population of paths over several generations to find a good solution.

1. The Problem Setup

Let's define a simple environment: a **10x10 grid**.

- **Start (S):** (0, 0)
- **Goal (G):** (9, 9)
- **Obstacles (X):** Solid blocks that the path cannot cross.

	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Example #2

2. GA Design

Chromosome Representation : A path will be represented by a sequence of moves.

- **Genes**: 0 (Up), 1 (Down), 2 (Left), 3 (Right).
- **Chromosome Length**: A direct path from (0,0) to (9,9) requires 18 moves (9 down, 9 right). Using chromosome length to allow for detours around obstacles. Let's set the **length to 25**.

Fitness Function : $\text{Fitness} = 1 / \text{Cost}$, where a lower cost is better.

$$\text{Cost} = (W_{\text{dist}} * \text{DistanceToGoal}) + (W_{\text{len}} * \text{PathLength}) + (W_{\text{col}} * \text{Collisions})$$

- **DistanceToGoal**: The Euclidean distance from the path's final point to the goal G(9,9). This is calculated as.
- **PathLength**: The *number of valid moves* made before hitting an obstacle or going off-grid.
- **Collisions**: The number of times the path hits an obstacle X or goes off the grid.
- **Weights (W)**: These balance the importance of each component. Heavily penalizing collisions.
 - $W_{\text{dist}} = 5$ (Strongly encourages getting closer to the goal)
 - $W_{\text{len}} = 1$ (Encourages shorter paths)
 - $W_{\text{col}} = 1000$ (Massive penalty for invalid moves)



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Example #2



3. (Generation 1) • Genes: 0 (Up), 1 (Down), 2 (Left), 3 (Right).

Create Initial Population (Generation 0) : 4 random chromosomes (paths).

- **C1:** [1, 3, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 3, 3, 3, 3, 3]
- **C2:** [3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 2, 0, 3, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0]
- **C3:** [1, 1, 1, 1, 3, 3, 3, 0, 0, 3, 1, 1, 1, 3, 3, 3, 1, 1, 1, 3, 3, 3, 3]
- **C4:** [3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]

	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Calculate Fitness (shown only for C2)

- **Path Trace for C2:** [3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 2, 0, 3, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0, 0]
 - S(0,0) -> R(0,1) -> R(0,2) -> R(0,3)
 - D(1,3) -> COLLISION! Path hits obstacle at (1,3).
- **Evaluation for C2:**
 - **PathLength:** 4 valid moves were made.
 - **Collisions:** 1.
 - **Final Point:** (1,3).
 - **DistanceToGoal:** $\sqrt{(3-9)^2 + (1-9)^2} = \sqrt{(-6)^2 + (-8)^2} = \sqrt{36+64} = \sqrt{100} = 10.0$.
 - **Cost:** $(5 * 10.0) + (1 * 4) + (1000 * 1) = 50 + 4 + 1000 = 1054$.
 - **Fitness:** $1 / 1054 = 0.000948$.

Example #2



- Genes: 0 (Up), 1 (Down), 2 (Left), 3 (Right).

Selection

Using Roulette Wheel selection, C1 has a massively higher probability of being chosen. For this example, let's select the two best parents:

- Parent 1: C1
- Parent 2: C2 (Even though it's bad, we select it to show the process. In a real run, C1 would likely be selected twice).

	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Crossover

Generate two offspring from Parent 1 and Parent 2 using single-point crossover. Random crossover point at **index 10**.

- Parent 1 (C1): [1, 3, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 3, 3, 3, 3, 3, 3]
- Parent 2 (C2): [3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 2, 0, 3, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0, 0, 0]

Higher fitness
- 0 collisions,
reaches the
goal.

Swap to create two children:

- Child 1: [1, 3, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 3, 3, 3, 3, 3, 3]
- Child 2: [3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 2, 0, 3, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0, 0, 0]

Chromosome	Final Point	Path Length	Collisions	Distance to Goal	Total Cost	
C1	(9, 9)	18	0	0	18	0.05555
C2	(1, 3)	4	1	10	1054	0.00095
C3	(1, 4)	5	1	9.43	1052.15	0.00095
C4	(3, 3)	6	1	8.48	1053.4	0.00095

Example #2



- **Genes:** 0 (Up), 1 (Down), 2 (Left), 3 (Right).

	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Mutation

A small random to each child to introduces new genetic material.

- **Mutating Child 1:** Randomly changed the gene at **index 12** from 3 (Right) to 1 (Down).
 - **Child 1 (Mutated):** [1, 3, 3, 1, 3, 1, 3, 1, 3, 1, 2, 0, ****1****, 1, 1, 1, 3, 3, 3, 3, 3, 0, 0, 0, 0]
- **Mutating Child 2:** Randomly changed the gene at **index 2** from 3 (Right) to 1 (Down).
 - **Child 2 (Mutated):** [3, 3, ****1****, 1, 1, 1, 1, 1, 1, 3, 3, 1, 3, 1, 3, 1, 3, 1, 3, 3, 3, 3, 3, 3]

Create New Population (Generation 1)

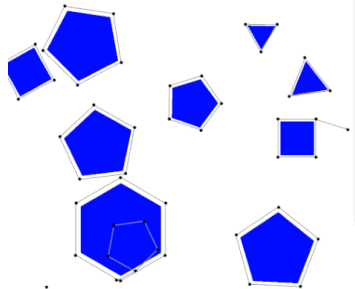
The two new, mutated children will now replace the two worst-performing individuals (C3 and C4) from the original population. This strategy is called **elitism**, as it guarantees the best individual (C1) survives.

- **New Population:** [C1, C2, Child 1 (Mutated), Child 2 (Mutated)]

The entire process (Fitness Calculation, Selection, Crossover, Mutation) is now repeated for this new generation. Over time, the fitness of the population will increase, evolving paths that are shorter, collision-free, and successfully reach the goal.

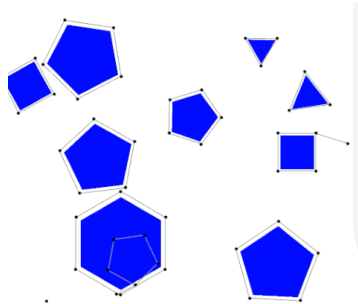
Optimization algorithms based Path Planning

Ant Colony Optimization Algorithm

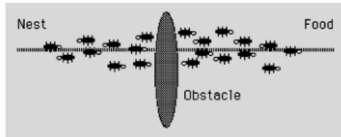


Recap from 1st Sem ACI course

Ant Colony Optimization Algorithm

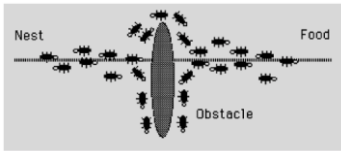


Ant Colony Optimization



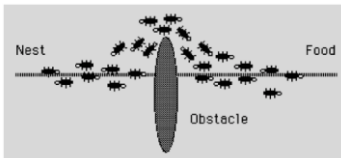
As ants have weak vision and weak memory, they communicate using the stimulus

They leave chemical deposits called **Pheromone** to inform other ants about food, colonies, etc.



Using olfactory **stimulus** the insects follow the **trail** left behind by other ants

Other ants follow the pheromone trail and **deposit even more pheromone** leading to a positive feedback effect, **if the selected path is good.**

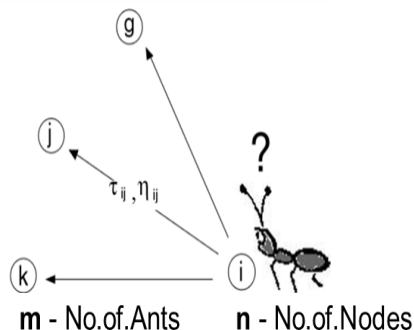


Ants **prefer** to follow a path with rich in pheromone

Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

Create Construction Graph
 Define & Initialize Pheromone Trails
 Define Heuristics
 Construct Solution
 Update Pheromones



S_k - Memory α - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

Cities = $\{i, j, k, g, \dots\}$

$\Delta\tau_{ij}^k$ is the pheromone laid on edge (i, j)
 by ant k

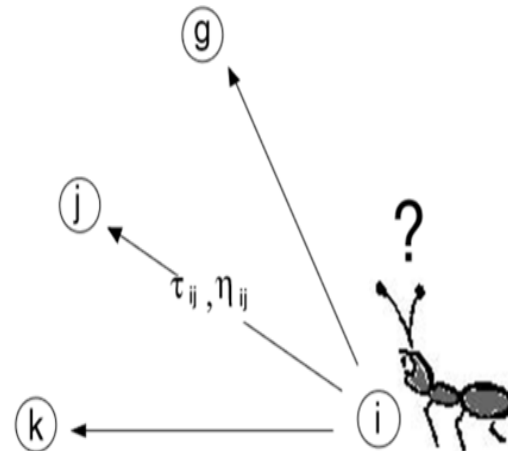
Ants move through states of problem by applying a stochastic local decision policy based on two parameters, **trails and attractiveness**

When an ant completes a solution, or during construction phase, the solution is evaluated and trail values on components are modified

Ant Colony Optimization – Recap from 1st Semester course ACI

At the core of ACO, an ant k moves from a state t to another state φ based on a probability $p_{t\varphi}^k$ which depends on combination of two values:

- Attractiveness ($\eta_{t\varphi}$) of the move, computer based on some heuristic indicating the prior desirability of that move
- Trail Level ($\tau_{t\varphi}$) of the move, indicating the usefulness of this move in the past



Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

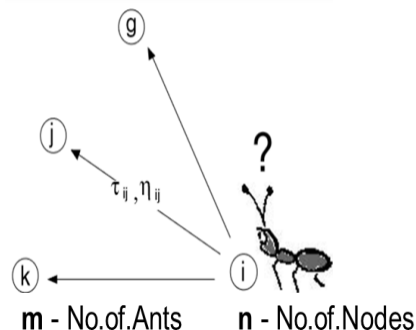
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution

Update Pheromones



S_k - Memory α - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

While real ants deposit chemical substance on the visited state, artificial ants change numerical information of problem state, stored locally when that state is visited

Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

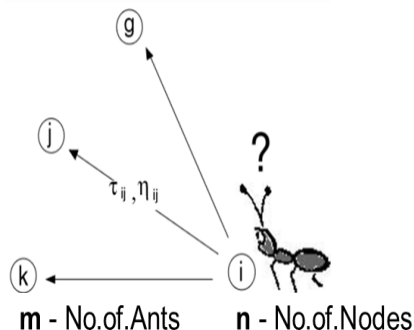
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution

Update Pheromones



S_k - Memory **α** - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

$$\tau_0 = \frac{m}{c} \qquad \tau_0 = \frac{1}{\rho c}$$

p - In ACO and real ants it is important to slowly forget the history to avoid stacking in local extremes and move toward new regions using the evaporation rate

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{if ant } k \text{ used edge } (i,j) \\ 0 & \text{otherwise} \end{cases}$$

where Q is a constant
 L_k is the length of tour constructed by ant k

Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

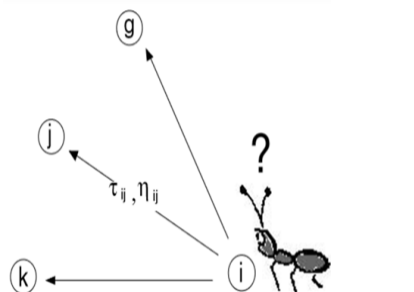
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution

Update Pheromones



m - No.of.Ants

n - No.of.Nodes

S_k - Memory

α - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

$$n_{i,j}(S_k) = \frac{1}{d_{i,j}}$$

d_{ij} is the distance between i and j

$n_{i,j}$: To optimize for shortest paths, every state transition from i to j , is guided by the attractiveness as the heuristic

Ant Colony Optimization – Recap from 1st Semester course ACI

innovate

achieve

lead

Problem Definition

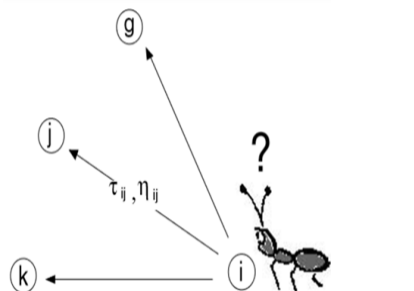
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution – Action Choice Rule

Update Pheromones



m - No.of.Ants **n** - No.of.Nodes

S_k - Memory **α** - Influence of τ

β - Influence of η

ρ – Pheromone evaporation rate

Way Point based path Planning

$$p_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{c_{ij} \in N(s^p)} \tau_{ij}^\alpha \eta_{ij}^\beta} & \text{if } c_{ij} \in N(s^p) \\ 0 & \text{otherwise} \end{cases}$$

Transition Policy: Decision making is based on probabilistic inference. It depends on local state information and existing pheromone trails

edge(*i*,*j*) where *j* is not yet visited $j \in \mathcal{N}_i^k$

Parameters α, β control relative importance of pheromone vs heuristic

Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

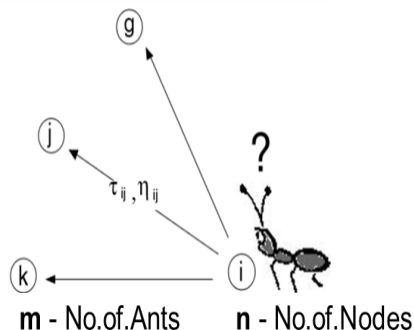
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution

Update Pheromones: Evaporation



S_k - Memory **α** - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad \forall (i, j) \in L$$

Local Update

$$\tau_{ij} = (1 - \varphi) * \tau_{ij} + \varphi * \tau_0$$

A Local pheromone update is performed by all ants only to the last edge it traversed after each construction step

$\varphi \in (0, 1)$ is pheromone decay coefficient

Global Update

$$\tau_{ij} \leftarrow \begin{cases} (1 - \rho) * \tau_{ij} + \rho * \Delta \tau_{ij} & \text{if } (i, j) \text{ belongs to best tour} \\ \tau_{ij} & \text{otherwise} \end{cases}$$

$$\Delta \tau_{ij} = 1/L_{best}$$

Where L_{best} - length of tour of best ant either in current iteration or best so far

Applied at the end of iteration by only one ant (either iteration-best/best-so-far)

Ant Colony Optimization – Recap from 1st Semester course ACI

Problem Definition

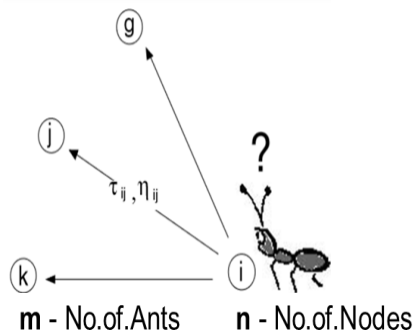
Create Construction Graph

Define & Initialize Pheromone Trails

Define Heuristics

Construct Solution

Update Pheromones: Deposit



S_k - Memory

α - Influence of τ

β - Influence of n

ρ - Pheromone evaporation rate

Way Point based path Planning

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}^{best},$$

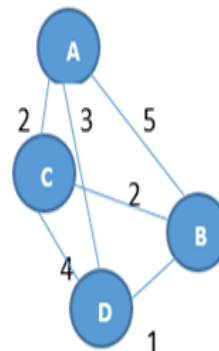
$$\text{where } \Delta\tau_{ij}^{best} = 1/C^{best}.$$

Deposited amount of Pheromone: Depends on quality of discovered food source / cost of solution found so far

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	1	1	1	A
	0	1	1	B
		0	1	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



No.of.Ants : $m=5$

Pheromone evaporation rate: $\rho=0.5$

α - Influence of τ = 1

β - Influence of n = 1

$\tau_{i,j}$ - Pheromone level = 1

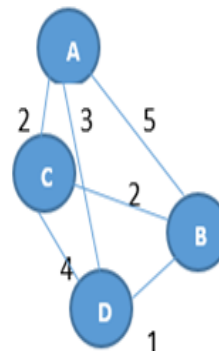
Optimal path finder for warehouse bots

1. Represent the solution space by a construction graph
2. Initialize ACO parameters
3. Generate random solutions from each ant's random walk
4. Update pheromone information
5. Go to 3 and repeat until convergence or a stopping criteria is met

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	1	1	1	A
	0	1	1	B
		0	1	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 1: 5 ants randomly sent

1: ABDCA → $C_k = AB + BD + DC + CA = 12$

2: ADBCA → $C_k = 8$

3: ABDCA → $C_k = 12$

4: ADBCA → $C_k = 8$

5: ACBDA → $C_k = 8$

This is a random trail .

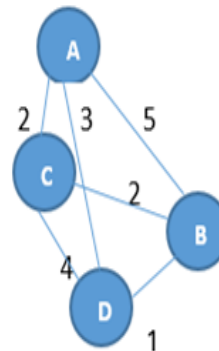
Typically the algorithm uses the probability value based Transition Policy for optimal search to construct the paths

1. Represent the solution space by a construction graph
2. Initialize ACO parameters
3. Generate random solutions from each ant's random walk
4. Update pheromone information
5. Go to 3 and repeat until convergence or a stopping criteria is met

Ant Colony Optimization - Sample Problem

A	B	C	D	
0	1 0.58	1	1	A
	0	1	1	B
		0	1	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 1: 5 ants

1: **ABDCA** → $C_k = 12$

2: **ADBCA** → $C_k = 8$

3: **ABDCA** → $C_k = 12$

4: **ADBCA** → $C_k = 8$

5: **ACBDA** → $C_k = 8$

$$\Delta \tau_{ij} = \frac{1}{C_k}$$

Pheromone Update

AB:

2 ants with $C_k = 12$

$$\text{ant 1: } \Delta \tau_{ij} = \frac{1}{12}$$

$$\text{ant 3: } \Delta \tau_{ij} = \frac{1}{12}$$

$$\text{AB: } \tau_{ij} \leftarrow (1 - \rho) * \tau_{ij} + \rho \sum_{k=1}^m \Delta \tau_{ij}^k$$

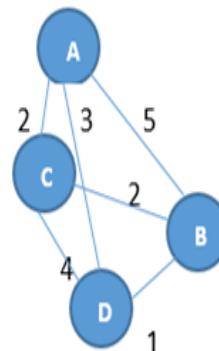
$$\tau_{ij} \leftarrow (1 - 0.5) * 1 + \left(\frac{1}{12} + \frac{1}{12} \right) * 0.5$$

$$\tau_{ij} \leftarrow 0.58$$

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	0.58	1.04	0.875	A
	0	0.875	1.04	B
		0	0.75	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 1: (Above table is a random values filled for rest.

Students should try to work out for the right values.)

1: ABDCA $\rightarrow C_k = 12$

2: ADBCA $\rightarrow C_k = 8$

3: ABDCA $\rightarrow C_k = 12$

4: ADBCA $\rightarrow C_k = 8$

5: ACBDA $\rightarrow C_k = 8$

$$\Delta \tau_{ij} = \frac{1}{C_k}$$

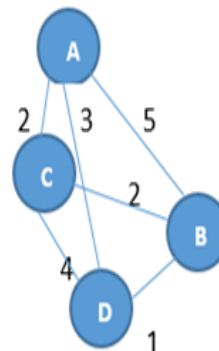
$$\tau_{ij} \leftarrow (1 - \rho) * \tau_{ij} + \rho \sum_{k=1}^m \Delta \tau_{ij}^k$$

1. Represent the solution space by a construction graph
2. Initialize ACO parameters
3. Generate random solutions from each ant's random walk
4. Update pheromone information
5. Go to 3 and repeat until convergence or a stopping criteria is met

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	0.58	1.04	0.875	A
	0	0.875	1.04	B
		0	0.75	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 2: 3 ANTS randomly sent

1: BCADB → $C_k = 8$

2: BDACB → $C_k = 8$

3: DBACD → $C_k = 12$

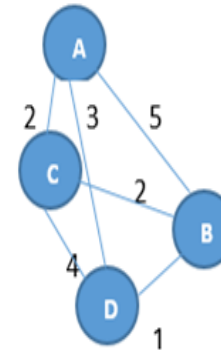
$$\tau_{ij} \leftarrow (1 - \rho) * \tau_{ij} + \rho \sum_{k=1}^m \Delta \tau_{ij}^k$$

$$\Delta \tau_{ij} = \frac{1}{C_k}$$

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	0.58	1.04	0.875	A
	0	0.875	1.04	B
		0	0.75	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 2: 3 ants

- 1: BCADB → $C_k = 8$
- 2: BDACB → $C_k = 8$
- 3: DBACD → $C_k = 12$

$$\Delta \tau_{ij} = \frac{1}{C_k}$$

Pheromone Update

AC:

3 ants with $C_k = 12, 8, 8$

$$\text{ant 1: } \Delta \tau_{ij} = \frac{1}{8}$$

$$\text{ant 2: } \Delta \tau_{ij} = \frac{1}{8}$$

$$\text{ant 3: } \Delta \tau_{ij} = \frac{1}{12}$$

$$\text{AC: } \tau_{ij} \leftarrow (1 - \rho) * \tau_{ij} + \rho \sum_{k=1}^m \Delta \tau_{ij}^k$$

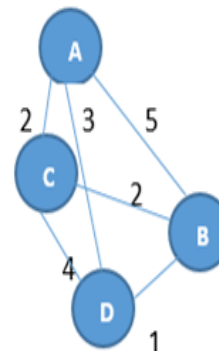
$$\tau_{ij} \leftarrow (1 - 0.5) * 1.04 + 0.5 \left(\frac{1}{8} + \frac{1}{8} + \frac{1}{12} \right)$$

$$\tau_{ij} \leftarrow 0.687$$

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	0.413	0.853	0.687	A
	0	0.687	0.853	B
		0	0.458	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Iteration 2: 3 ANTS

1: BCADB → $C_k = 8$

2: BDACB → $C_k = 8$

3: DBACD → $C_k = 12$

(Above table is a random values filled for rest.
Students should try to work out for the right values.)

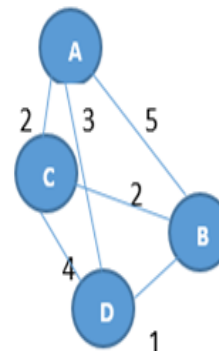
$$\tau_{ij} \leftarrow (1 - \rho) * \tau_{ij} + \rho \sum_{k=1}^m \Delta \tau_{ij}^k$$

$$\Delta \tau_{ij} = \frac{1}{C_k}$$

Ant Colony Optimization – Recap from 1st Semester course ACI

A	B	C	D	
0	0.413	0.853	0.687	A
	0	0.687	0.853	B
		0	0.458	C
			0	D

A	B	C	D	
0	5	2	3	A
	0	2	1	B
		0	4	C
			0	D



Solution Choice:

A Bot arrives at a waypoint C. Which is the next hop ?

$$CA := \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{C_{ij} \in N(s^p)} \tau_{ij}^\alpha \eta_{ij}^\beta} = \frac{0.853 * (\frac{1}{2})}{(0.853 * \frac{1}{2} + 0.687 * \frac{1}{2} + 0.458 * \frac{1}{4})} \sim 60 - 70\%$$

Similarly calculate for below edges as well before deciding the next node to send

BOTS to.

CB:=

CD:=

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \quad \text{if } j \in \mathcal{N}_i^k$$

ACO as path planning (way-point Optimization)



1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

ACO as path planning (way-point Optimization)

3 layers of 3 candidate via-points (A/B/C) = 11 nodes, 24 edges, 27 candidate tours. Pheromone now lives on layer-to-layer edges.

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

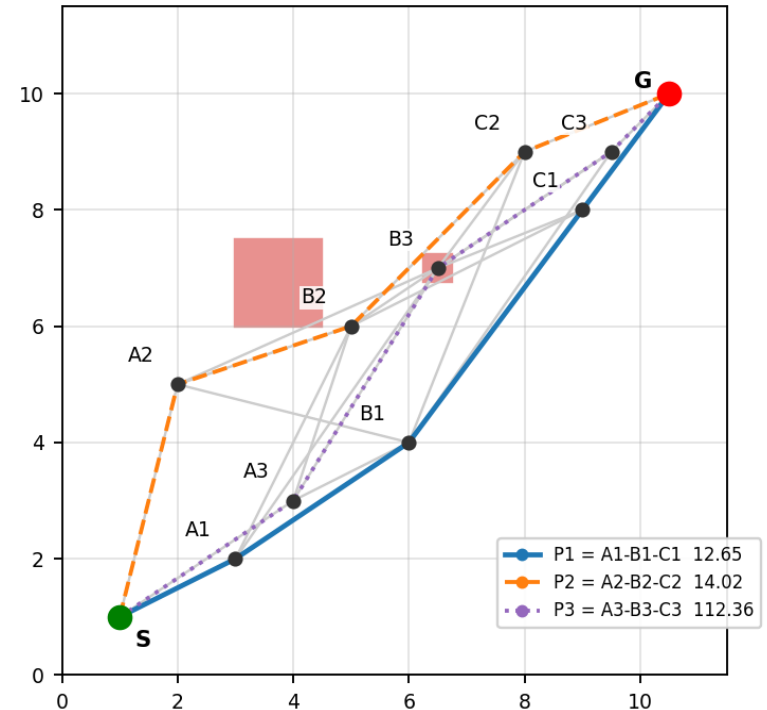
Start S: (1, 1) **Goal G:** (10.5, 10)

ACO needs a graph, so the continuous way-point space is discretized into candidate waypoints.

Layer A (first level): A1(3,2), A2(2,5), A3(4,3)
Layer B (second): B1(6,4), B2(5,6), B3(6.5,7)
Layer C (third): C1(9,8), C2(8,9), C3(9.5,9)

Each ant builds a path $S \rightarrow A_i \rightarrow B_j \rightarrow C_k \rightarrow G$, so the bot's paths become e.g., $P1 = A1-B1-C1$, $P2 = A2-B2-C2$, $P3 = A3-B3-C3$.

Discretised via-point graph (3 layers)

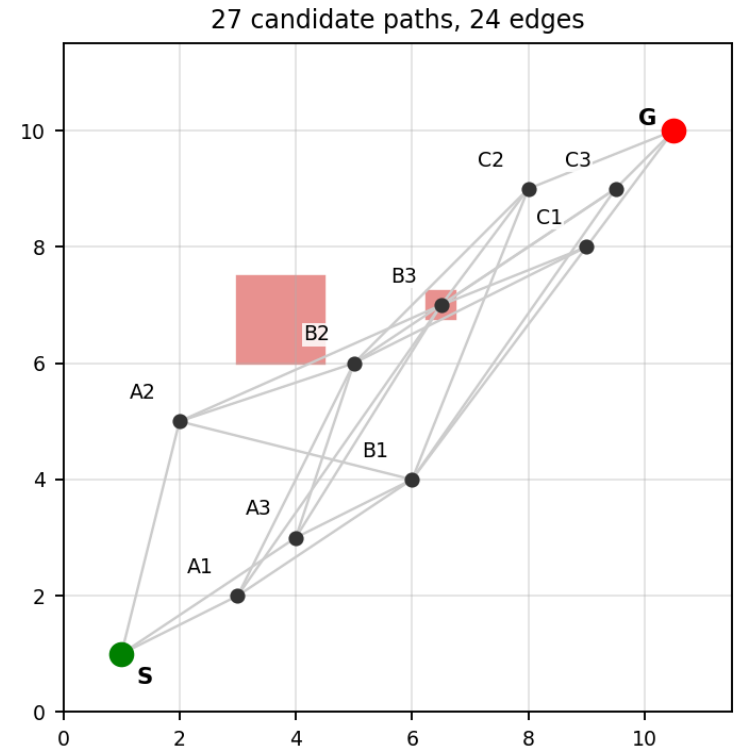


ACO as path planning (way-point Optimization)

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

Start S: (1, 1) Goal G: (10.5, 10)

- **Pheromone $\tau(i,j)$:** learned desirability of using j after i.
- **Heuristic $\eta(i,j) = 1 / d(j, G)$:** static
- $\tau_0 = 0.1$ on all 24 edges
- **Colony size $m = 5$ ants per iteration**
- An edge whose straight segment cuts the obstacle is still selectable, but the resulting path pays the 100 penalty and deposits nothing.



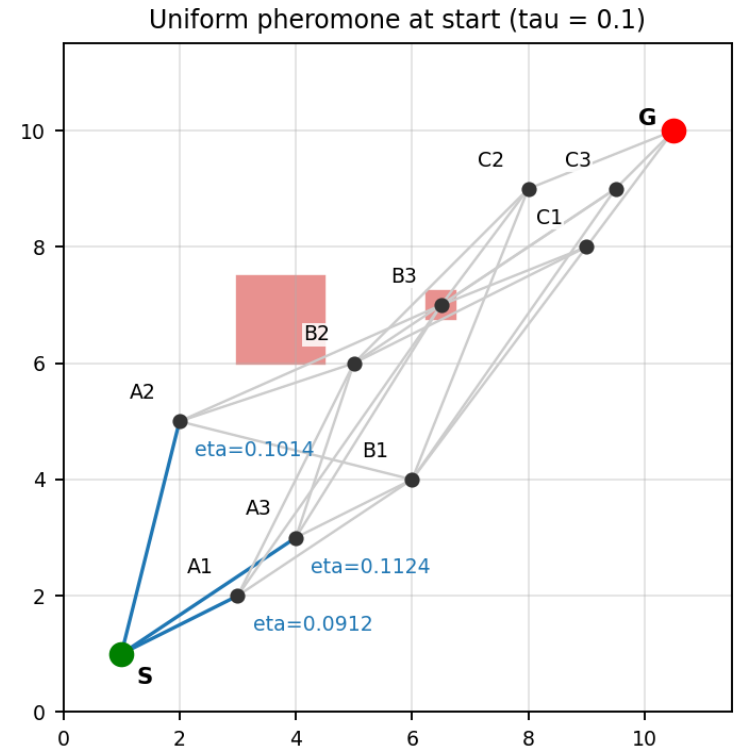
ACO as path planning (way-point Optimization)

A3 initially looks the most attractive even though it leads to the colliding path

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

Start S: (1, 1) Goal G: (10.5, 10)

- Heuristic $\eta(i,j) = 1 / d(j, G)$: static
- **A1(3,2):** $d = \sqrt{(7.5^2 + 8^2)} = 10.97 \Rightarrow \eta = 0.0912$
- **A2(2,5):** $d = \sqrt{(8.5^2 + 5^2)} = 9.86 \Rightarrow \eta = 0.1014$
- **A3(4,3):** $d = \sqrt{(6.5^2 + 7^2)} = 8.90 \Rightarrow \eta = 0.1124$



ACO as path planning (way-point Optimization)

Hyperparameter : $\alpha = 1$ (pheromone influence): high α exploits known trails, risks premature convergence. $\beta = 1$ (heuristic influence): high β makes ants greedy toward the goal and blind to obstacles behind it. $\rho = 0.5$ (evaporation): controls how fast an obsolete trail is forgotten.

1. Initialize ants, pheromone, heuristics
2. **Construct Graph (Path) & Evaluate**
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

Start S: (1, 1) **Goal G:** (10.5, 10)

Ant standing at S, iteration 1 (all $\tau = 0.1$, $\alpha = 1$, $\beta = 1$)

S→A1: $0.1 \times 0.0912 = 0.00912$

S→A2: $0.1 \times 0.1014 = 0.01014$

S→A3: $0.1 \times 0.1124 = 0.01124$

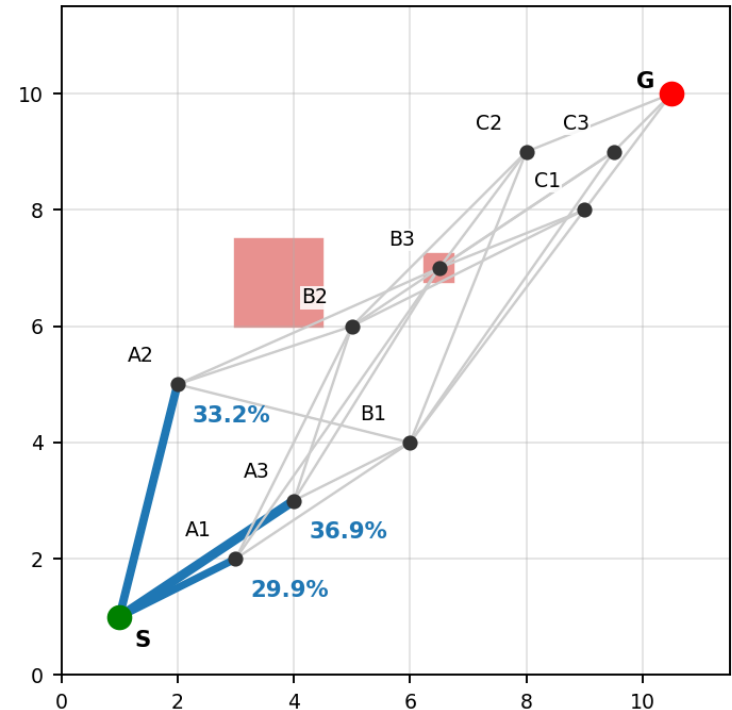
$$P_{ij} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\sum_{k \in \text{allowed neighbors}} [\tau_{ik}]^{\alpha} \cdot [\eta_{ik}]^{\beta}}$$

P(S→A1) = 29.9 %

P(S→A2) = 33.2 %

P(S→A3) = 36.9 %

Iteration 1: first-move probabilities



ACO as path planning (way-point Optimization)

Ant 5 is a genuinely new path that no firefly ever generated — it comes from mixing layers, which is the effect of probabilistic construction

1. Initialize ants, pheromone, heuristics
2. **Construct Graph (Path) & Evaluate**

The 5 ants complete their paths

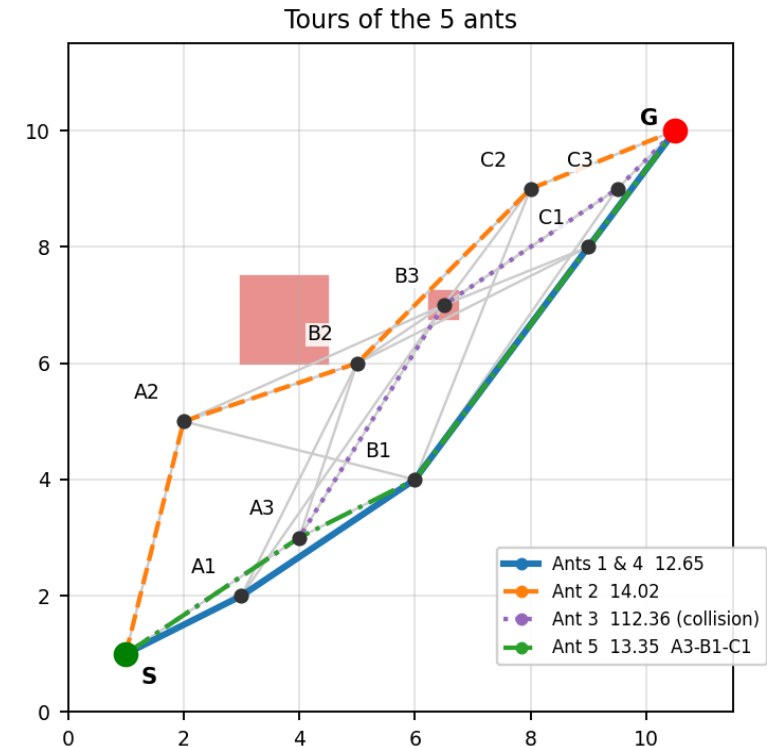
Ant 1: S-A1-B1-C1-G = (1,1),(3,2),(6,4),(9,8),(10.5,10) $\Rightarrow L = 12.65$

Ant 2: S-A2-B2-C2-G = (1,1),(2,5),(5,6),(8,9),(10.5,10) $\Rightarrow L = 14.02$

Ant 3: S-A3-B3-C3-G = (1,1),(4,3),(6.5,7),(9.5,9),(10.5,10) $\Rightarrow L = 12.36$

Ant 4: S-A1-B1-C1-G — same route as Ant 1 $\Rightarrow L = 12.65$

Ant 5: S-A3-B1-C1-G = (1,1),(4,3),(6,4),(9,8),(10.5,10)
 $\Rightarrow L = 3.61 + 2.24 + 5.00 + 2.50 = 13.35$



ACO as path planning (way-point Optimization)

Deposit amount uses $\Delta\tau = Q / \text{Cost}$, so a cheaper tour writes a stronger trail

1. Initialize ants, pheromone, heuristics
2. **Construct Graph (Path) & Evaluate**
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

Ant 1: Cost = 12.65 + 0 = **12.65**

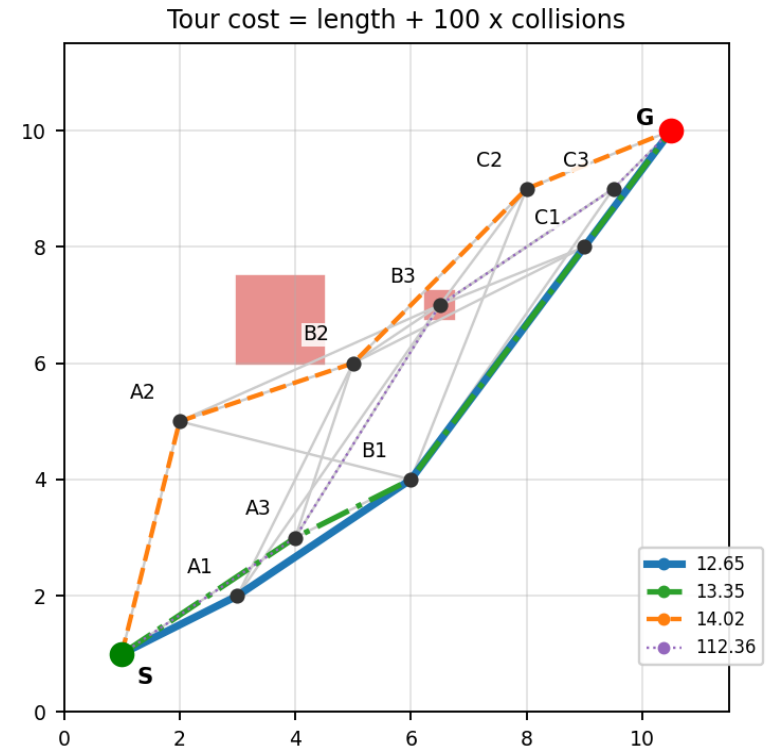
Ant 2: Cost = 14.02 + 0 = **14.02**

Ant 3: Cost = 12.36 + 100 = **112.36** → treated as an invalid tour, deposits nothing

Ant 4: Cost = **12.65**

Ant 5: Cost = **13.35**

Iteration best = 12.65 (Ants 1 and 4)



ACO as path planning (way-point Optimization)

Evaporation is the colony's forgetting mechanism — without it the first lucky trail would dominate for ever and the search would freeze.

Low ρ (0.1): long memory, slow to abandon a poor early route. High ρ (0.8): fast forgetting, close to a random restart each iteration.

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. **Evaporate pheromone**
4. Deposit pheromone
5. Iterate next trail until convergence

- $\tau(i,j) \leftarrow (1 - \rho) \cdot \tau(i,j)$, with $\rho = 0.5$.
- Every one of the 24 edges falls from 0.1 to **0.05** before any deposit is added.

First, the colony “forgets”

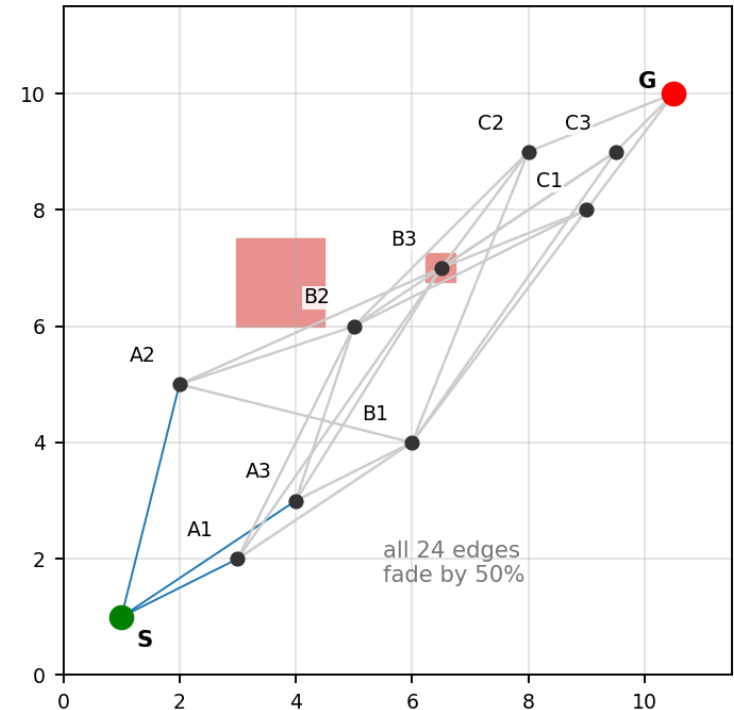
Evaporation rate $\rho = 0.5$ applied to every edge:

$$\tau(i,j) \leftarrow (1 - \rho) \cdot \tau(i,j)$$

Every one of the 24 edges drops 0.10 $\rightarrow$ 0.05

Without evaporation, the first lucky trail would dominate forever and the colony could never re-route.

Evaporation: every edge 0.10 $\rightarrow$ 0.05



tion) imagine achieve lead

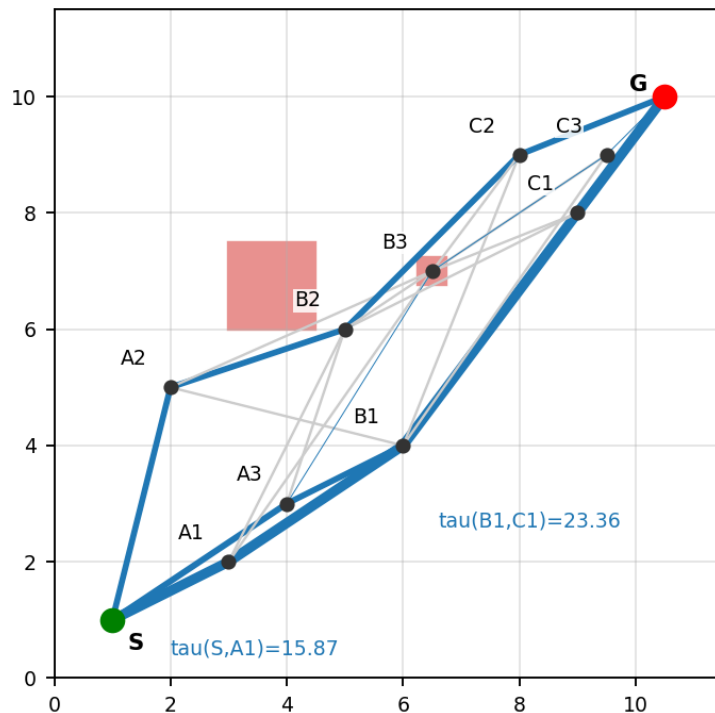
ACO is the only one of the three with an explicit, decaying memory stored on the environment rather than in the population

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. **Deposit pheromone**
5. Iterate next trail until convergence

Deposit $\Delta\tau = Q / \text{Cost}$, $Q = 100$

- **Ant 1 (12.65):** $\Delta_T = 100 / 12.65 = 7.91$ on edges S-A1, A1-B1, B1-C1, C1-G
- **Ant 2 (14.02):** $\Delta_T = 100 / 14.02 = 7.13$ on S-A2, A2-B2, B2-C2, C2-G
- **Ant 4 (12.65):** $\Delta_T = 7.91$ on the same edges as Ant 1
- **Ant 5 (13.35):** $\Delta_T = 100 / 13.35 = 7.49$ on S-A3, A3-B1, B1-C1, C1-G
- **Ant 3:** invalid tour $\Rightarrow$ deposits **0**

Pheromone after deposition (line width $\sim \tau$)



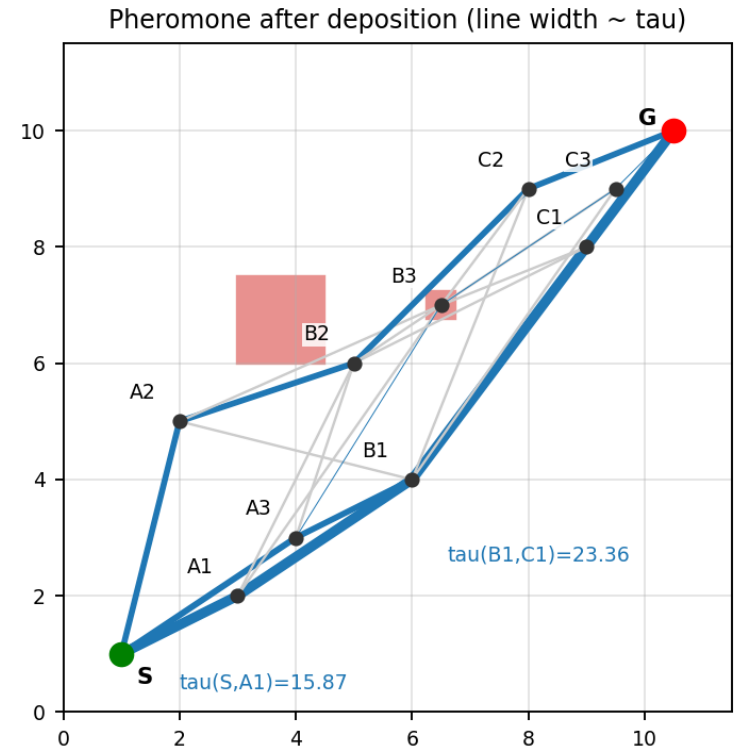
ACO as path planning (way-point Optimization)

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
- 4. Deposit pheromone**
5. Iterate next trail until convergence

Deposit $\Delta\tau = Q / \text{Cost}$, $Q = 100$

Updated pheromone on the S-layer edges

- $\tau(S,A1) = 0.05 + 7.91 + 7.91 = 15.87$ $\tau(S,A2) = 0.05 + 7.13 = 7.18$
 $\tau(S,A3) = 0.05 + 7.49 = 7.54$
- $\tau(B1,C1) = 0.05 + 7.91 + 7.91 + 7.49 = 23.36$ — the most reinforced edge, shared by three good tours.



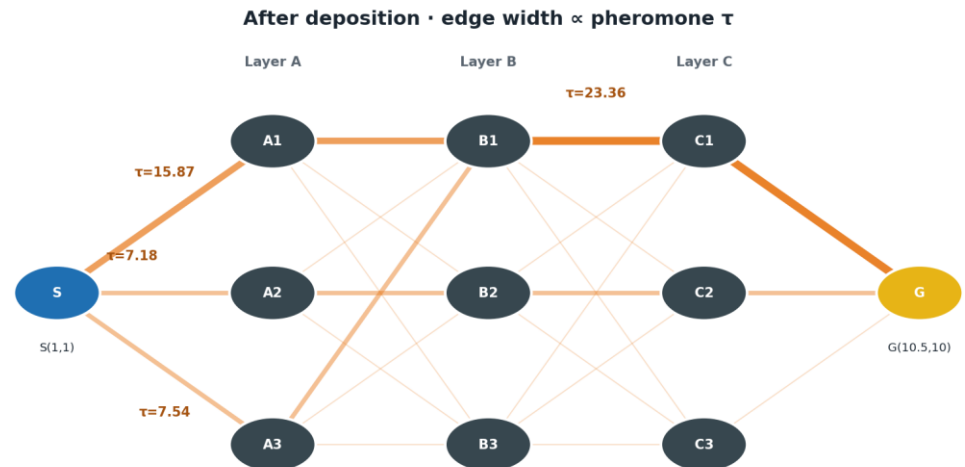
ACO as path planning (way-point Optimization)

Firefly uses brightness, GA uses fitness-proportionate selection
ACO uses a pheromone trail shared across iterations

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. **Deposit pheromone**
5. Iterate next trail until convergence

m = 5 ants and iteration count: more ants give a smoother pheromone update per iteration.

Practical : start with high β for exploration, then raise α to exploit



Evaporation $0.1 \rightarrow 0.05$ on all 24 edges, then $\Delta\tau = Q/\text{Cost}$ added on valid tours (Ant 3 deposits 0).

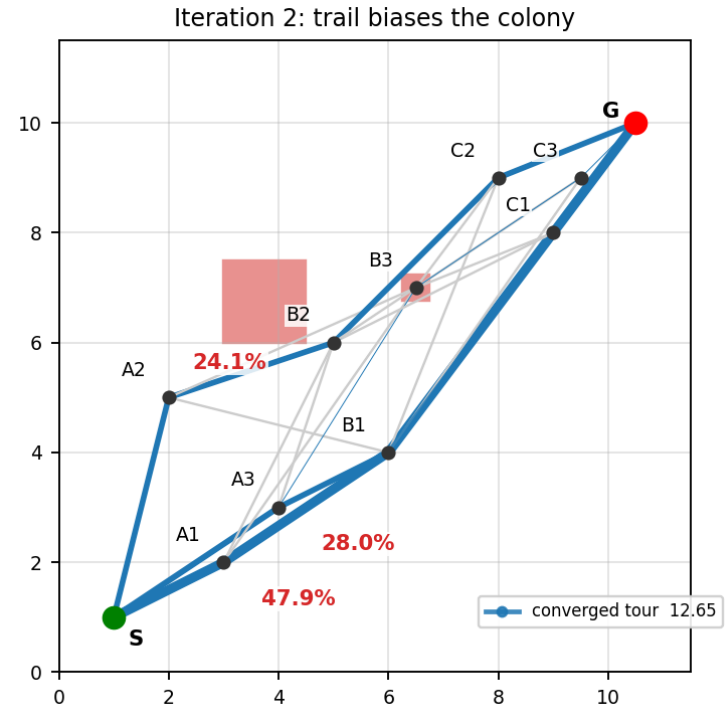
ACO as path planning (way-point Optimization)

Convergence is declared when the best tour is unchanged and one edge per layer holds most of the pheromone mass

1. Initialize ants, pheromone, heuristics
2. Construct Graph (Path) & Evaluate
3. Evaporate pheromone
4. Deposit pheromone
5. Iterate next trail until convergence

Biased transition probabilities from S

- $S \rightarrow A1$: $15.87 \times 0.0912 = 1.447$
- $S \rightarrow A2$: $7.18 \times 0.1014 = 0.728$
- $S \rightarrow A3$: $7.54 \times 0.1124 = 0.848$
- $P(A1) = 47.9\%$, $P(A2) = 24.1\%$, $P(A3) = 28.0\%$
- A1 has risen from 29.9 % to 47.9 % in a single iteration — this is the colony learning, and it is what replaces the firefly's attraction step.
- **Trend over iterations (illustrative)**: iteration 1 best 12.65, iteration 5 best 12.65 with $P(A1) \approx 78\%$, iteration 10 $P(A1) > 90\%$ — the colony has converged on S-A1-B1-C1-G.



Learning to Adapt ACO Parameters (α, β, ρ)

The performance of ACO is highly sensitive to its parameters: α (pheromone influence), β (heuristic influence), and ρ (evaporation rate). These are usually fixed after manual tuning. However, the best strategy often changes during the search.

- **Initial Phase:** It's better to encourage **exploration** (ants trying new paths), which means a higher β value.
- **Later Phase:** Once good paths are found, it's better to encourage **exploitation** (ants following the strong pheromone trails), which means a higher α value.
- **How to incorporate learning:** Use **Reinforcement Learning (RL)** to create a self-tuning algorithm.
 1. **Create an RL Agent:** An RL agent is tasked with adjusting the ACO parameters.
 2. **Define State, Action, Reward:**
 - **State:** The current state could be defined by metrics like the current iteration number, the improvement rate of the best path, or the diversity of the paths found by the ants.
 - **Action:** The agent's actions are to slightly increase or decrease α , β , and ρ .
 - **Reward:** The agent receives a positive reward if its action leads to a faster discovery of a better solution in the next few iterations.
 3. **Dynamic Adjustment:** Over time, the RL agent learns a policy for how to best adjust the ACO parameters on the fly for any given problem.

Function AntColonyOptimization:

Initialize all edges with a small pheromone value τ_0 .

Loop for N iterations:

// 1. Ant Construction Phase

For each Ant k in m:

Place Ant k at Start node.

While Ant k has not reached Goal node:

Calculate transition probabilities P_{ij} for all allowed neighbor nodes j.

Select next node j based on probabilities.

Move Ant k to node j and record the edge.

Store the path and its length L_k .

// 2. Pheromone Update Phase

For each edge (i, j) in the graph:

// Evaporation

$\tau_{ij} = (1 - \rho) * \tau_{ij}$

For each Ant k that found a path:

// Deposition

AmountToDeposit = Q / L_k

For each edge (i, j) in Ant k's path:

$\tau_{ij} = \tau_{ij} + \text{AmountToDeposit}$

// Return the best path found across all iterations.

BestPath = FindBestPathFound()

Return BestPath

- ACO cannot beat the resolution of its graph — finer discretization gives better paths at higher computational cost.
- **Best use:** ACO for grid or roadmap graphs and repeated queries on a static map; GA and firefly for continuous refinement

Additional Practice problem Added

Parameters used in ACO

General pseudo-code

Procedure ACO

Schedule Activities

Initialization

Construction

Update Pheromone

// local search, elitism

End schedule activities

End ACO

Parameter	Description
N	Total No of ants ; $N > 1$
τ_o	Initial pheromone amount
τ_{ij}	Amount of pheromone deposited while traversing from i to j
η_{ij}	$1/(\text{Cost of link (i,j)})$
α	Importance coefficient of pheromone intensity
β	Importance coefficient of route cost
ρ	evaporation co-efficient; $0 < \rho < 1$
$visit_k$	Visited nodes table of k^{th} ant
Q	Importance - Constant value pertaining to pheromone trail
f_k	Route cost obtained by ant k

Core Formulas

- **Transition Probability:** The probability of an ant at node i choosing to move to an adjacent node j .

$$P_{ij} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\sum_{k \in \text{allowed neighbors}} [\tau_{ik}]^{\alpha} \cdot [\eta_{ik}]^{\beta}}$$

- **Pheromone Update:** After all ants complete their paths, the pheromones on all edges are updated.

Evaporation: $\tau_{ij}^{new} = (1 - \rho)\tau_{ij}^{old} + \Delta\tau_{ij}^k$

Deposition: For each ant k that used edge (i, j) in its path of length L_k given as f_k , the pheromone is increased:

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{f_k} & \text{if } k^{th} \text{ ant passes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

Example #2



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Over time, shorter paths get reinforced more quickly, leading the colony to converge on the optimal route.

The Problem Setup

Let's define a simple environment: a **10x10 grid**.

- **Start (S):** (0, 0)
- **Goal (G):** (9, 9)
- **Obstacles (X):** Solid blocks that the path cannot cross.
- **Pheromones (τ_o):** Every edge (i, j) on the graph has a pheromone value τ_j . Initially, we'll set this to a small, constant value for all edges: $\tau_o=0.1$.
- **Heuristic Information (η_{ij}):** A "greedy" value that makes nodes closer to the goal more attractive. It's the inverse of the distance from the next node j to the goal G . $\eta_{ij}=1/\text{distance}(j,G)$
- **Ants:** A population of m agents that traverse the graph from S to G. Let $m = 5$ ants.
- **Parameters:**
 - $\alpha=1$: Controls the influence of pheromones.
 - $\beta=1$: Controls the influence of the heuristic.
 - $\rho=0.5$: The pheromone evaporation rate (50% evaporates each iteration).
 - $Q=100$: A pheromone deposit constant.

Example #2



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Ant Path Construction

All 5 ants start at $S(0, 0)$. Compute the decision for the first move of an ant.

- The ant is at $(0, 0)$. Allowed neighbors are $(1, 0)$ (Down) and $(0, 1)$ (Right)
- **Initial Pheromone:** $\tau_{((0,0),(1,0))}=0.1$ and $\tau_{((0,0),(0,1))}=0.1$
- **Heuristic Calculation:**
 - η_{ij} for $(1, 0)$: Distance to $G(9, 9) = \sim 12.04$. So, $\eta_{ij} = 1/12.04 = 0.083$
 - η_{ij} for $(0, 1)$: Distance to $G(9, 9) = \sim 12.04$. So, $\eta_{ij} = 1/12.04 = 0.083$
- **Transition Probability Calculation:** (Since $\alpha=1$, $\beta=1$)
 - **P(move to $(1,0)$)** = $0.0083/0.0166=0.5$ (50%)
 - **P(move to $(0,1)$)** = $0.0083/0.0166=0.5$ (50%)

Example #2



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

At the start, the choice is random.

As pheromones change, this will become biased.

Each of the 5 ants makes a probabilistic choice and continues until it reaches the goal or gets stuck.

Assume after this process, our 5 ants found the following paths:

Ant	Path Length (f_k)	Successfully Reached Goal?
Ant 1	22	Yes
Ant 2	28	Yes
Ant 3	-	No (stuck in a dead end)
Ant 4	24	Yes
Ant 5	22	Yes

Example #2



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	.	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

Pheromone Update

Evaporation: First, the pheromone on *every single edge* in the graph evaporates by 50% ($\rho=0.5$).

- $\tau_{new}=(1-0.5)$
- So, all edges now have a pheromone level of 0.05

Deposition: Next, the successful ants (1, 2, 4, 5) deposit new pheromones on the paths they took.

- **Pheromone deposited by Ant 1 (L=22):** $\Delta\tau = Q/L_1 = 100/22 = 4.55$
- **Pheromone deposited by Ant 2 (L=28):** $\Delta\tau = Q/L_2 = 100/28 = 3.57$
- **Pheromone deposited by Ant 4 (L=24):** $\Delta\tau = Q/L_4 = 100/24 = 4.17$
- **Pheromone deposited by Ant 5 (L=22):** $\Delta\tau = Q/L_5 = 100/22 = 4.55$. (Note: Ant 3 deposits nothing as it failed)

Final pheromone level on a few edges:

- **An edge used only by Ant 1:** $\tau_{final} = (\text{evaporated value}) + (\text{Ant1's_deposit}) = 0.05 + 4.55 = 4.60$
- **An edge used by both Ant 1 and Ant 5 (shorter paths):**
 $\tau_{final} = (\text{evaporated value}) + (\text{Ant1}) + (\text{Ant5}) = 0.05 + 4.55 + 4.55 = 9.15$
- **An edge used by no successful ant:** $\tau_{final} = (\text{evaporated value}) + 0 = 0.05$

Ant	Path Length (f_k)
Ant 1	22
Ant 2	28
Ant 3	-
Ant 4	24
Ant 5	22

Example #2



	0	1	2	3	4	5	6	7	8	9
0	S	.	.	.	.	.	.	.	.	.
1	.	.	.	X	X	X	X	.	.	.
2	.	.	.	.	.	.	X	.	.	.
3	.	.	.	.	.	.	X	.	.	.
4	.	.	.	X	X	X	X	.	.	.
5	.	.	.	X	.	.	.	.	.	.
6	.	.	.	X	.	.	.	.	.	.
7	.	.	.	X	.	X	X	X	X	X
8	.	.	.	.	.	X	.	.	.	.
9	.	.	.	.	.	.	.	.	.	G

After 1st iteration

- The pheromone map is no longer uniform.
- Edges that were part of shorter, successful paths (like the one used by Ants 1 and 5) now have significantly higher pheromone levels (9.15) than unused edges (0.05).

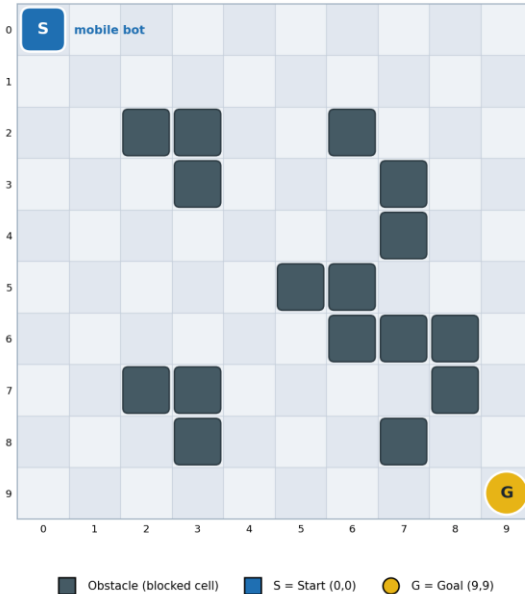
In **Iteration 2**, when a new set of ants starts from $S(0, 0)$, their probability calculation will be heavily biased towards these reinforced paths, guiding them to better solutions.

This cycle repeats for a set number of iterations.

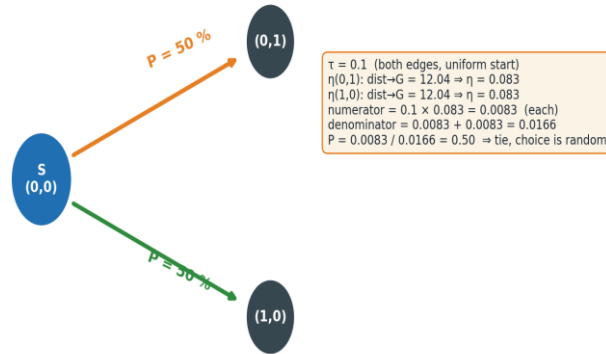
ACO Demo — Grid Setup (Mobile Bot)

$\tau_0 = 0.1$ on every edge · $\eta(j)=1/\text{dist}(j,G)$ · $m = 5$ ants · $\alpha=1, \beta=1, \rho=0.5, Q=100$

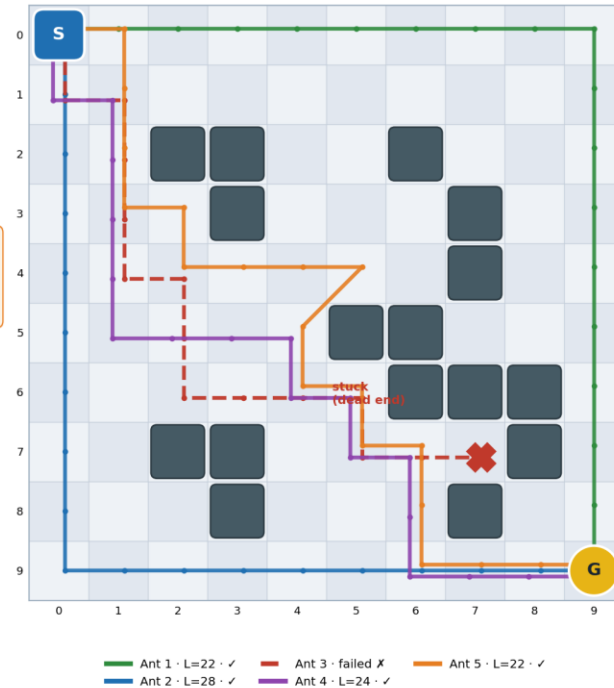
ACO live run · 10x10 grid · S(0,0) → G(9,9)



Step 1 · First transition probability from S(0,0)



Ant tour construction · 5 ants explore S → G



Knowledge lives in the environment, not the population

GA and Firefly store knowledge in the individuals (the population). ACO instead writes its knowledge into the environment as pheromone trails — this indirect coordination is called stigmergy.

Built-in adaptivity:

- If an obstacle on the map were to move, evaporation would erase the stale trail while ants naturally re-route around the new geometry — no restart, no re-encoding required.

ACO Demo — Grid Setup (Mobile Bot)



Pheromone update — the colony learns

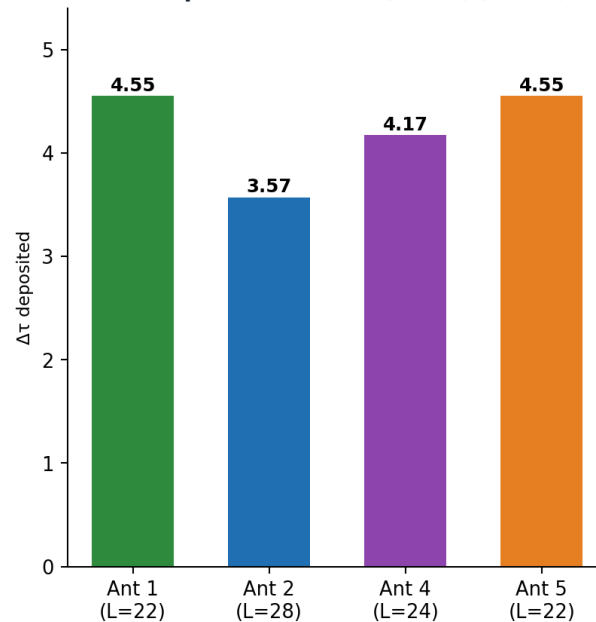
1 · Evaporation ($\rho = 0.5$)

$$\tau_{\text{new}} = (1 - \rho) \cdot \tau$$



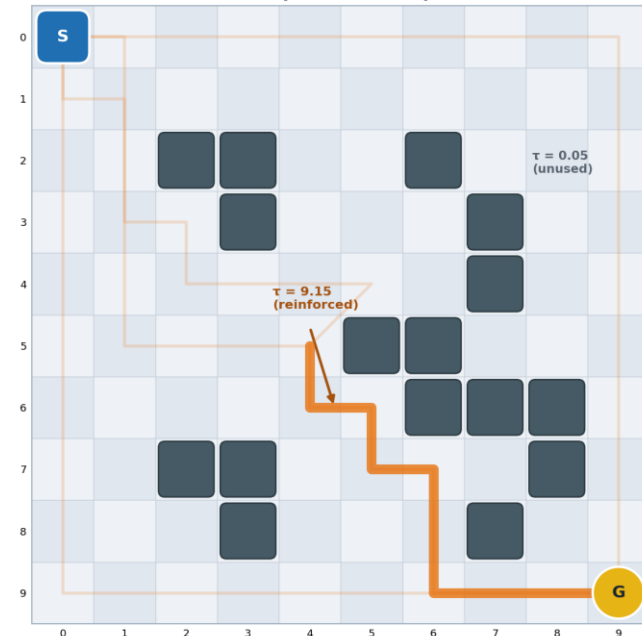
Every edge in the graph loses 50 % of its pheromone before any deposit.

2 · Deposition $\Delta\tau = Q / L$ ($Q=100$)



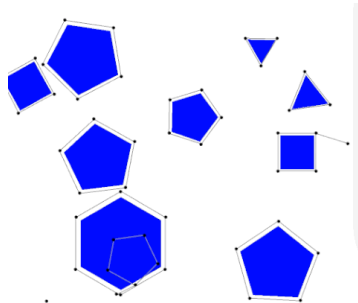
Iteration-2 ants are strongly biased toward the 9.15 corridor $\Rightarrow$ faster convergence.

After iteration 1 · pheromone map is now biased



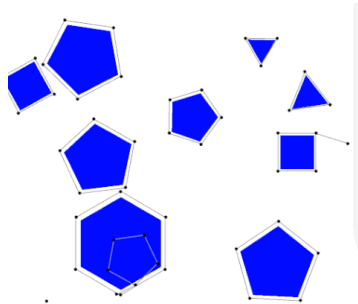
Optimization algorithms based Path Planning

Particle Swarm Optimization Algorithm



Recap from 1st Sem ACI course

Particle Swarm Optimization Algorithm

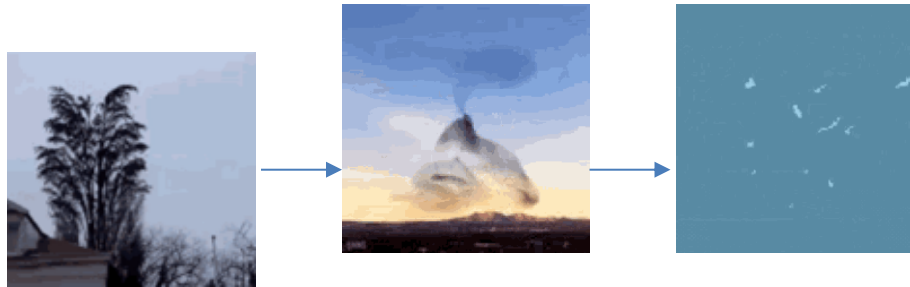


Particle Swarm Optimization

innovate

achieve

lead



Each individual is called a particle

Particle = Position vector $x_i(t)$ + Velocity vector $v_i(t)$

Heuristics Considered = function (Individual's Best So-far , Group's Best So-far)

Learning:

Adjusting the velocity of each particle toward its **pbest** and **lbest** locations

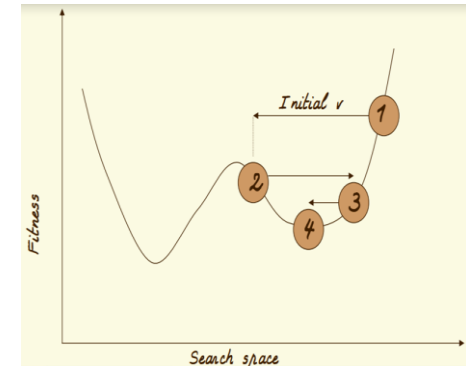
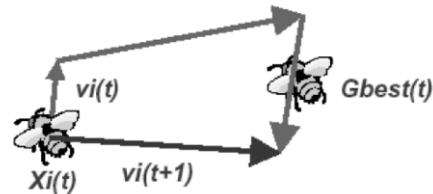
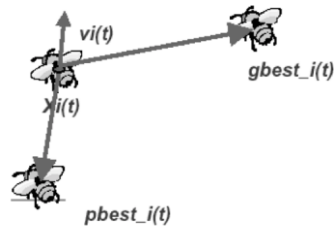
1. Global best (pbest): Known to all and immediately updated when new position is found by any particle
2. Local best / neighborhood best (lbest): Each particle communicates only with a subset of swarm about best positions.

Particle Swarm Optimization

innovate

achieve

lead



Updating velocity vector:

$$X_i(t + 1) = X_i(t) + V_i(t + 1)$$

$$v_i(t + 1) = \alpha v_i + c_1 \times rand \times (pbest(t) - x_i(t)) + c_2 \times rand \times (gbest(t) - x_i(t))$$

α is inertia weight and controls exploration and exploitation

c_1 and c_2 the cognition and social components respectively

rand is a random number generator

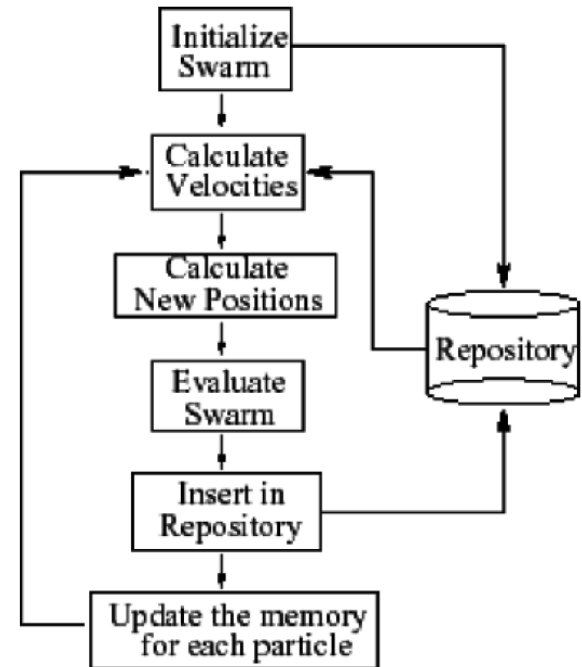
Particle Swarm Optimization



Basic Flow of PSO

1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify $\mathbf{g}_{best}$, $\mathbf{p}_{best}$, and velocity
4. Move each particle to new particle
5. Goto step 2, and repeat until convergence

Particles velocities on each dimension
are clamped to a max velocity v_{max}



PSO as path planning (way-point Optimization)



1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify $\mathbf{g}_{\text{best}}$, $\mathbf{p}_{\text{best}}$, and velocity → Particles velocities on each dimension are clamped to a max velocity v_{max}
4. Move each particle to new particle
5. Repeat until convergence

PSO as path planning (way-point Optimization)

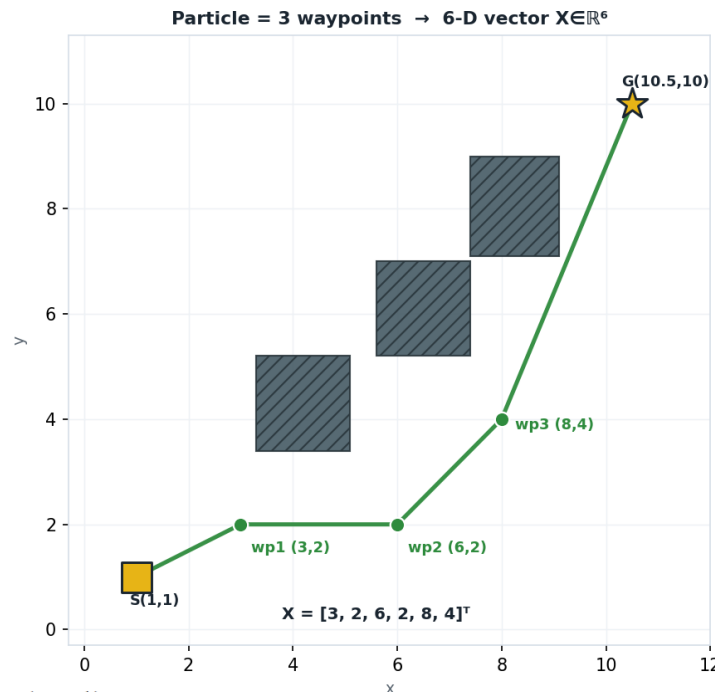
1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify $\mathbf{g}_{\text{best}}$, $\mathbf{p}_{\text{best}}$, and velocity
4. Move each particle to new particle
5. Repeat until convergence

Path as a vector of waypoints

- Each particle encodes 3 free waypoints $\rightarrow \mathbf{X}_i \in \mathbb{R}^6 = [x_1, y_1, x_2, y_2, x_3, y_3]^T$
- Fixed endpoints: Start $\mathbf{S}(1,1) \rightarrow$ Goal $\mathbf{G}(10.5,10)$.

Swarm

- 4 particles search the free space
- $W=\alpha=0.5$
- $c_1=c_2=1.5$
- $r_1, r_2 \sim U(0,1)^6$.



$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \mathbf{V}_i(t+1)$$

$$v_i(t+1) = \alpha v_i + c_1 \times rand \times (pbest(t) - x_i(t)) + c_2 \times rand \times (gbest(t) - x_i(t)) \quad 86$$

PSO as path planning (way-point Optimization)

1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify $\mathbf{g}_{\text{best}}$, $\mathbf{p}_{\text{best}}$, and velocity
4. Move each particle to new particle
5. Repeat until convergence

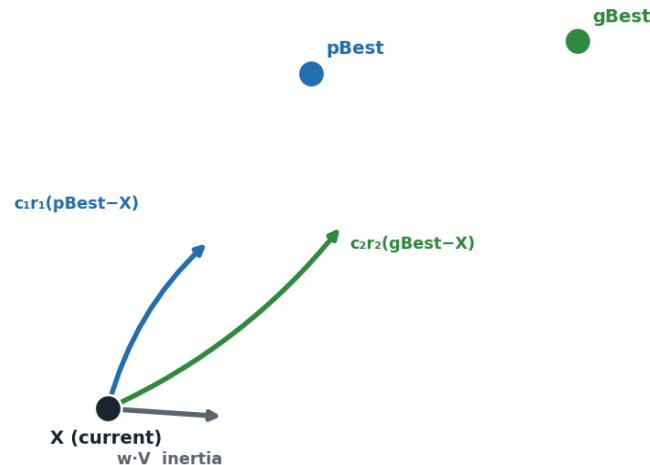
Velocity update = inertia + cognitive + social

Path as a vector of waypoints

- Each particle encodes 3 free waypoints $\rightarrow \mathbf{X}_i \in \mathbb{R}^6 = [x_1, y_1, x_2, y_2, x_3, y_3]^T$
- Fixed endpoints: Start **S(1,1)** $\rightarrow$ Goal **G(10.5,10)**.

Swarm

- 4 particles search the free space
- $W=\alpha=0.5$
- $c_1=c_2=1.5$
- $r_1, r_2 \sim U(0,1)^6$.



$$X_i(t+1) = X_i(t) + V_i(t+1)$$

$$v_i(t+1) = \alpha v_i + c_1 \times rand \times (pbest(t) - x_i(t)) + c_2 \times rand \times (gbest(t) - x_i(t)) \quad 87$$

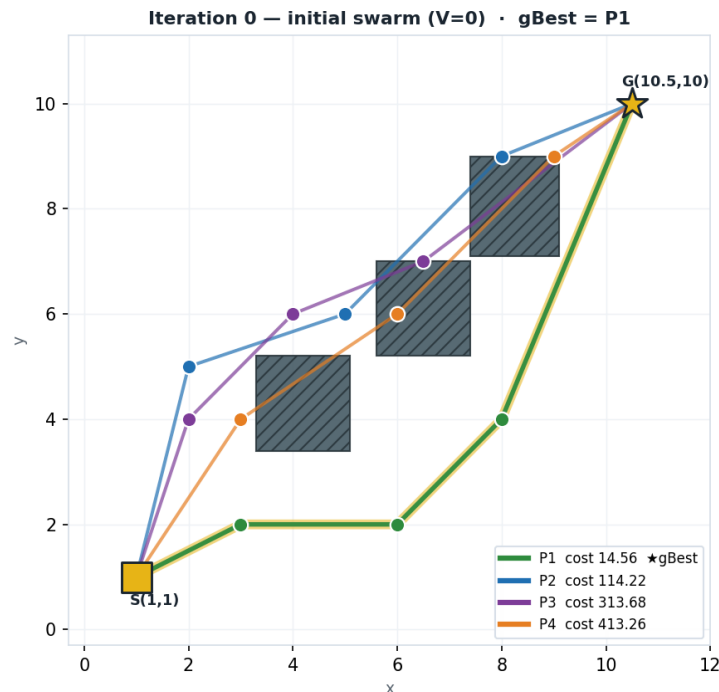
PSO as path planning (way-point Optimization)

1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify $\mathbf{g}_{best}$, $\mathbf{p}_{best}$, and velocity
4. Move each particle to new particle
5. Repeat until convergence

Iteration 0 — Initial Swarm ($V = 0$)

Four candidate paths = Particles

- P1 : $(1,1) \rightarrow (3,2) \rightarrow (6,2) \rightarrow (8,4) \rightarrow (10.5,10)$
- P2 : $(1,1) \rightarrow (2,5) \rightarrow (5,6) \rightarrow (8,9) \rightarrow (10.5,10)$
- P3 : $(1,1) \rightarrow (2,4) \rightarrow (4,6) \rightarrow (6.5,7) \rightarrow (10.5,10)$
- P4 : $(1,1) \rightarrow (3,4) \rightarrow (6,6) \rightarrow (9,9) \rightarrow (10.5,10)$



PSO as path planning (way-point Optimization)

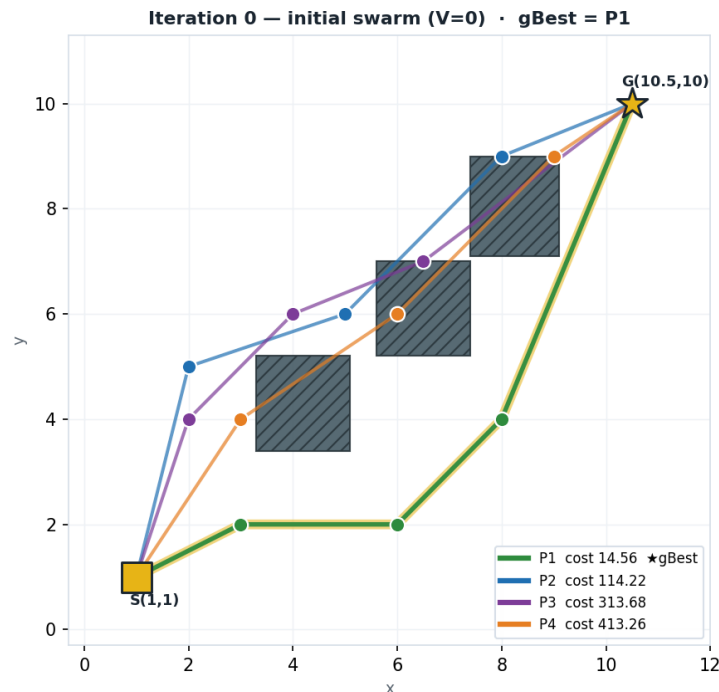
1. Initialize the swarm with random initializations
2. **Evaluate fitness value for each of these individuals**
3. Modify $\mathbf{g}_{best}$, $\mathbf{p}_{best}$, and velocity
4. Move each particle to new particle
5. Repeat until convergence

Iteration 0 — Initial Swarm ($V = 0$)

Four candidate paths = Particles

- P1 : $L = 14.56$, **0 collisions** → **cost 14.56**
- P2 : $L = 14.22$, 1 collision → **cost 114.22**
- P3 : $L = 13.68$, 3 collisions → **cost 313.68**
- P4 : $L = 13.26$, 4 collisions → **cost 413.26**

Global best
 $\mathbf{g}_{Best} = \mathbf{P1}$
The only collision-free trajectory
(cost 14.56)



$$\text{Fitness} = \text{Fitness}(P) = 1 / \text{Cost}(P)$$

$$\text{Cost}(P) = \sum \text{segment length} + 100 \times (\text{number of colliding segments})$$

PSO as path planning (way-point Optimization)

P1 : (1,1)→(3,2)→(6,2)→(8,4)→(10.5,10)

P4 : (1,1)→(3,4)→(6,6)→(9,9)→(10.5,10)

$W=\alpha=0.5$

$c_1=c_2=1.5$

$r_1, r_2 \sim 0.5$

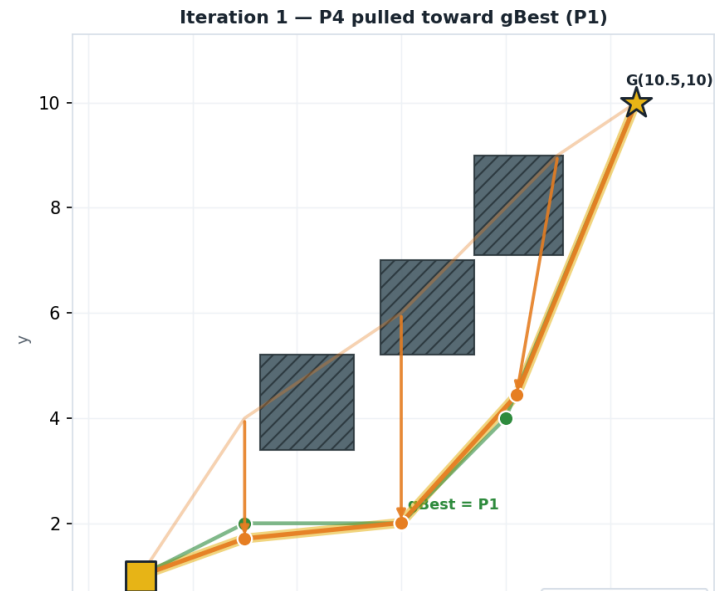
1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. **Modify g_{best} , p_{best} , and velocity**
4. Move each particle to new position
5. Repeat until convergence

Iteration 0 — Initial Swarm ($V = 0$)

Cognitive term = 0 ($p_{best} = X_0$) : With no personal history yet, motion is driven purely by the pull to g_{best} (P1).

Only one update : P4 (Students must practice for P_i updates)

- $g_{best}-X_4 = [0, -2, 0, -4, -1, -5]$
- $V_4^{(1)} = [0, -1.5, 0, -3, -0.75, -3.75]$
- $X_4^{(1)} = [3, 2.5, 6, 3, 8.25, 5.25]$
- Length 14.44, 0 collisions → cost 13.99 ★ **new g_{best}**



$$X_i(t+1) = X_i(t) + V_i(t+1)$$

$$v_i(t+1) = \alpha v_i + c_1 \times rand \times (pbest(t)-x_i(t)) + c_2 \times rand \times (gbest(t)-x_i(t))$$

PSO as path planning (way-point Optimization)

1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify g_{best} , p_{best} , and velocity
4. **Move each particle to new particle**
5. Repeat until convergence

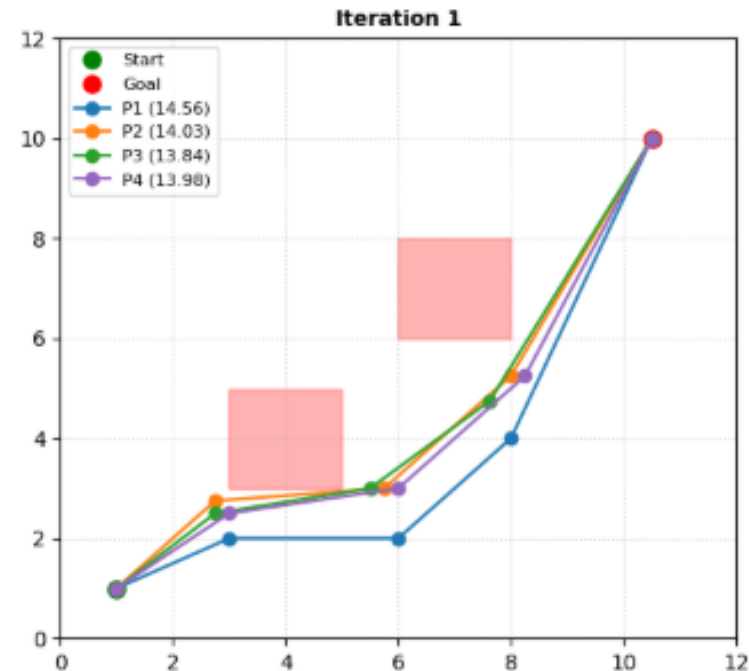
Iteration 0 — Initial Swarm ($V = 0$)

Positions & costs after update

- P1 [3, 2, 6, 2, 8, 4] → cost 14.56
- P2 [2.75, 2.75, 5.75, 3, 8, 5.25] → cost 14.03
- P3 [2.75, 2.5, 5.5, 3, 7.625, 4.75] → cost 13.84
- **P4 [3, 2.5, 6, 3, 8.25, 5.25] → cost 13.98.**

New global best

- **gBest = P4 (13.98)**



$$X_i(t+1) = X_i(t) + V_i(t+1)$$

$$v_i(t+1) = \alpha v_i + c_1 \times rand \times (pbest(t) - x_i(t)) + c_2 \times rand \times (gbest(t) - x_i(t))$$

PSO as path planning (way-point Optimization)

Waypoints are nudged as a continuous vector – the swarm converges on one smooth, collision-free region without any graph or encoding

1. Initialize the swarm with random initializations
2. Evaluate fitness value for each of these individuals
3. Modify g_{best} , p_{best} , and velocity
4. Move each particle to new particle
5. Repeat until convergence

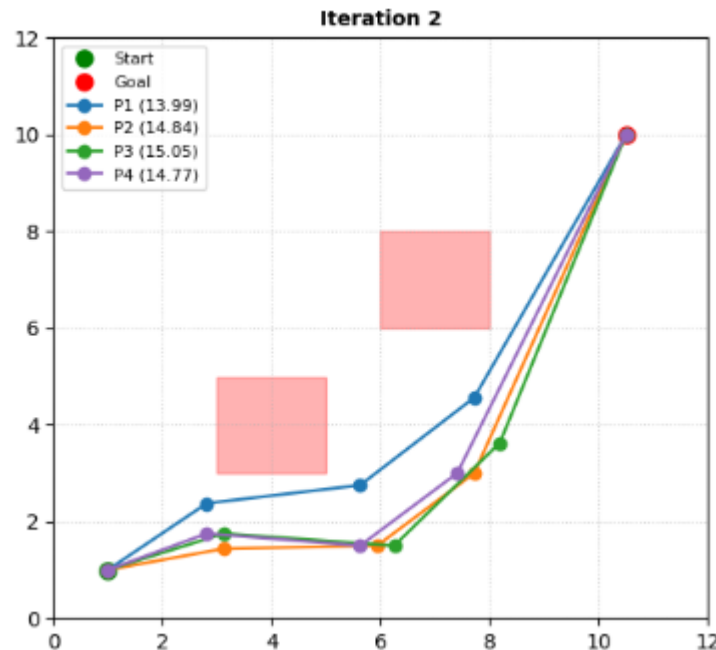
Iteration 0 — Initial Swarm ($V = 0$)

Positions & costs after update

- P1 [3, 2, 6, 2, 8, 4] → cost 14.56
- P2 [2.75, 2.75, 5.75, 3, 8, 5.25] → cost 14.03
- P3 [2.75, 2.5, 5.5, 3, 7.625, 4.75] → cost 13.84
- **P4 [3, 2.5, 6, 3, 8.25, 5.25] → cost 13.98.**

New global best

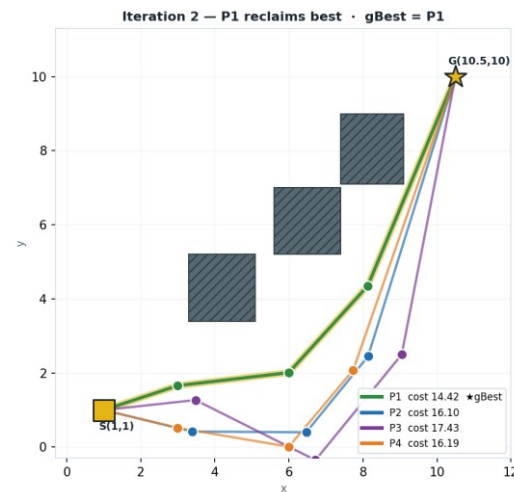
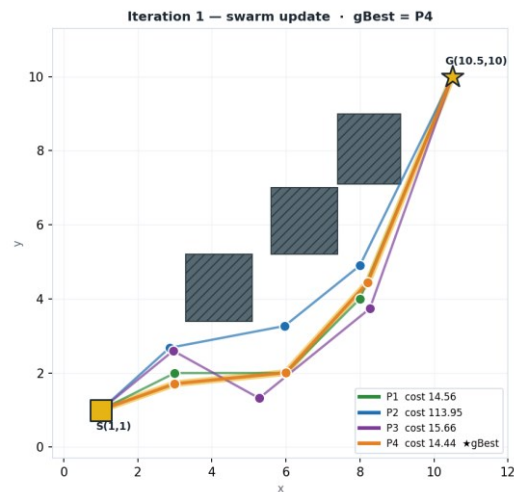
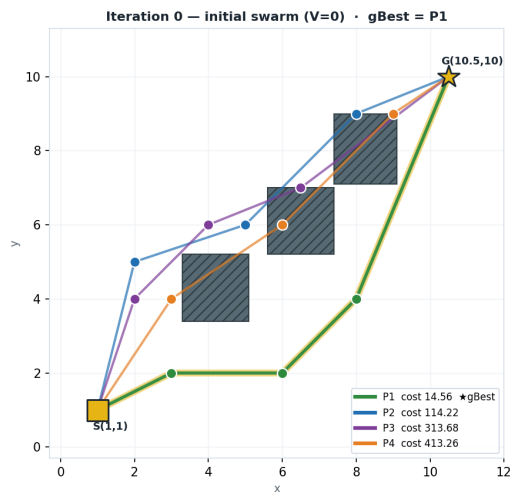
- **gBest = P4 (13.98)**



$$X_i(t+1) = X_i(t) + V_i(t+1)$$

$$v_i(t+1) = \alpha v_i + c_1 \times rand \times (pbest(t) - x_i(t)) + c_2 \times rand \times (gbest(t) - x_i(t))$$

Code Demo Results (with ri randomized)



Comparing Planners



Planner Type	Optimality	Completeness	Speed	Best for High Dimensions
Graph-Based	Optimal	Complete	Slow	No
Sampling-Based	Sub-optimal (RRT) / Near-optimal (RRT*)	Probabilistically	Fast	Yes
Optimization-Based	Optimal (local)	Incomplete	Varies	Yes
Learning-Based	Sub-optimal	Incomplete	Very Fast	yes
Hybrid	Depends on combo	Depends on combo	Fast & Reactive	Yes

Required Reading

- <https://erc-bpgc.github.io/handbook/>

Research Papers and Web Resources

Next Class Plan

✓ Evolutionary Algorithms in Robotics

✓ Overview of HRI

✓ Overview of integration of Vision and Language Models

Disclaimer : Given AV / NL integration warrants a separate study the next module will only be at an overview level. Interested readers who want to learn deep can go up to 3D-vision integration with depth perception, SLM with Generative features for sophisticated RI applications, AV for streaming app on edge devices, AIML Optimization for embedded

system

